Advanced Visualization Movie Ratings

In [2]:
```
import pandas as pd
pd.__version__                  # check version of pandas
```

Out[2]:
```
'2.2.2'
```

In [3]:
```
movies = pd.read_csv(r"C:\Users\Avinash\Downloads\Movie-Rating.csv")  # read the fi
```

In [4]:
```
movies
```

Out[4]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million $)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

In [5]:
```
type(movies)          # type of data structure in file movies
```

Out[5]:
```
pandas.core.frame.DataFrame
```

In [6]:
```
len(movies)           # no. of rows
```

Out[6]:
```
559
```

In [7]:
```
movies.columns        # column list
```

Out[7]:
```
Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %',
       'Budget (million $)', 'Year of release'],
      dtype='object')
```

In [8]:
```
movies.info()          # quick summery of dataframe
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column                    Non-Null Count  Dtype 
---  ------                    --------------  ----- 
 0   Film                      559 non-null    object
 1   Genre                     559 non-null    object
 2   Rotten Tomatoes Ratings %  559 non-null    int64 
 3   Audience Ratings %        559 non-null    int64 
 4   Budget (million $)        559 non-null    int64 
 5   Year of release           559 non-null    int64 
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

In [9]:
```
movies.shape           # row x column
```

Out[9]:
```
(559, 6)
```

In [10]:
```
movies.head()          # top rows; 5 rows by default
```

Out[10]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million $)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [11]:
```
movies.tail()          # last rows; 5 by default
```

Out[11]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million $)	Year of release
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

In [12]:
```
movies.columns
```

Out[12]:
```
Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %',
       'Budget (million $)', 'Year of release'],
      dtype='object')
```

In [13]:
```
# change name of columns

movies.columns = ['Film', 'Genre', 'CriticRating','AudienceRating','BudgetMillions'
```

In [14]:
```
movies.head(1)
```

Out[14]:

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009

In [15]:
```
movies.shape
```

Out[15]:
```
(559, 6)
```

In [16]:
```
movies.describe()    # descriptive statistics only numerical data
```

Out[16]:

	CriticRating	AudienceRating	BudgetMillions	Year
count	559.000000	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136	2009.152057
std	26.413091	16.826887	48.731817	1.362632
min	0.000000	0.000000	0.000000	2007.000000
25%	25.000000	47.000000	20.000000	2008.000000
50%	46.000000	58.000000	35.000000	2009.000000
75%	70.000000	72.000000	65.000000	2010.000000
max	97.000000	96.000000	300.000000	2011.000000

In [17]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   Film            559 non-null    object
 1   Genre           559 non-null    object
 2   CriticRating    559 non-null    int64 
 3   AudienceRating  559 non-null    int64 
 4   BudgetMillions  559 non-null    int64 
 5   Year            559 non-null    int64 
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

In [18]:
```
movies.Film = movies.Film.astype('category')  # change dtype of column values
```

In [19]:
```
movies.info()      # quick summery of dataframe
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype   
---  ------          --------------  -----   
 0   Film            559 non-null    category
 1   Genre           559 non-null    object  
 2   CriticRating    559 non-null    int64   
 3   AudienceRating  559 non-null    int64   
 4   BudgetMillions  559 non-null    int64   
 5   Year            559 non-null    int64   
dtypes: category(1), int64(4), object(1)
memory usage: 43.6+ KB
```

In [20]:
```
movies.Genre = movies.Genre.astype('category')
```

In [21]:
```
movies.Year = movies.Year.astype('category')
```

In [22]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype   
---  ------          --------------  -----   
 0   Film            559 non-null    category
 1   Genre           559 non-null    category
 2   CriticRating    559 non-null    int64   
 3   AudienceRating  559 non-null    int64   
 4   BudgetMillions  559 non-null    int64   
 5   Year            559 non-null    category
dtypes: category(3), int64(3)
memory usage: 36.5 KB
```

In [23]:
```
movies.describe()       # only numerical value data info
```

Out[23]:

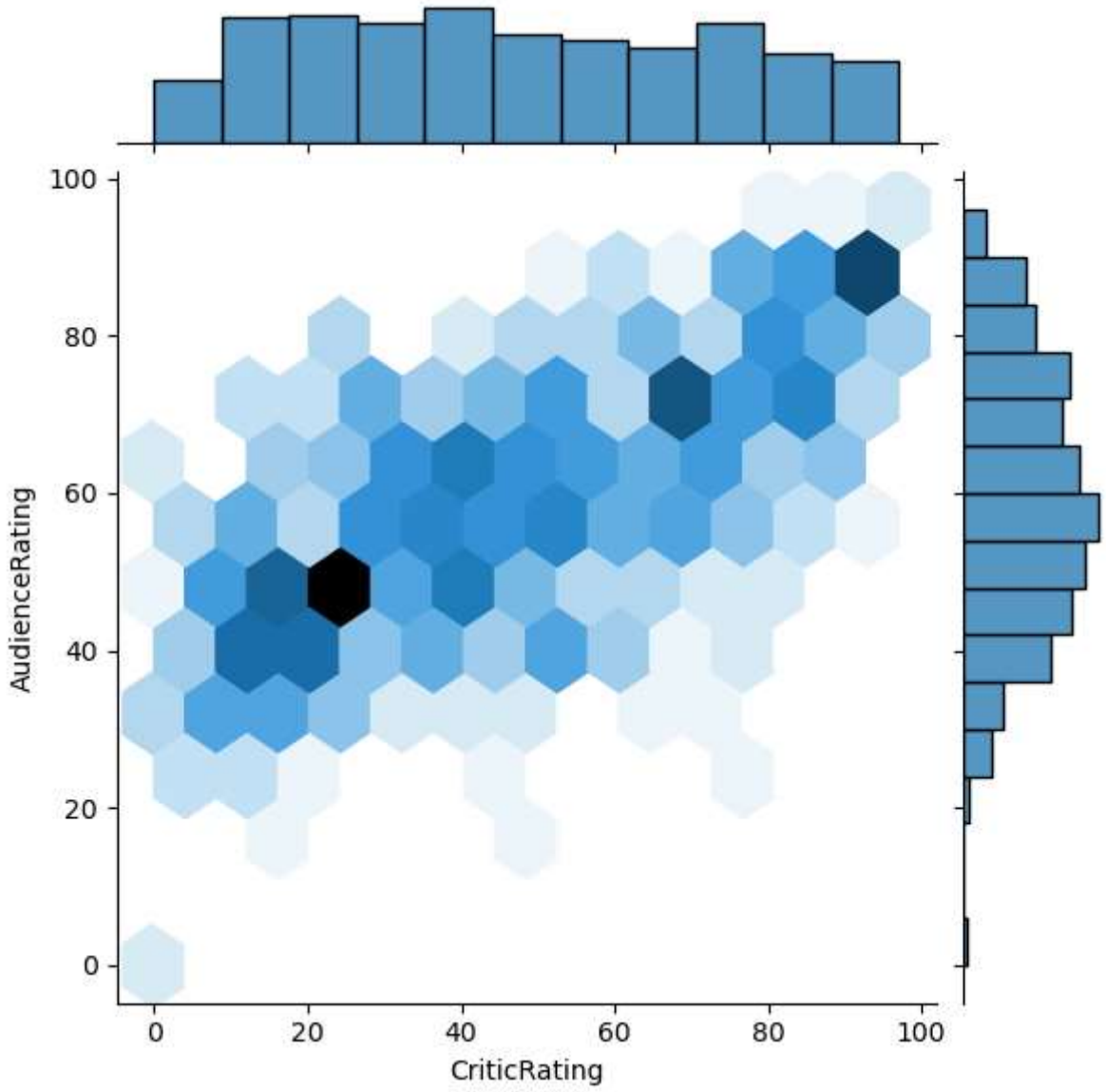
	CriticRating	AudienceRating	BudgetMillions
count	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136
std	26.413091	16.826887	48.731817
min	0.000000	0.000000	0.000000
25%	25.000000	47.000000	20.000000
50%	46.000000	58.000000	35.000000
75%	70.000000	72.000000	65.000000
max	97.000000	96.000000	300.000000

In [24]:
```
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

import warnings
warnings.filterwarnings('ignore')            # ignore warnings
```

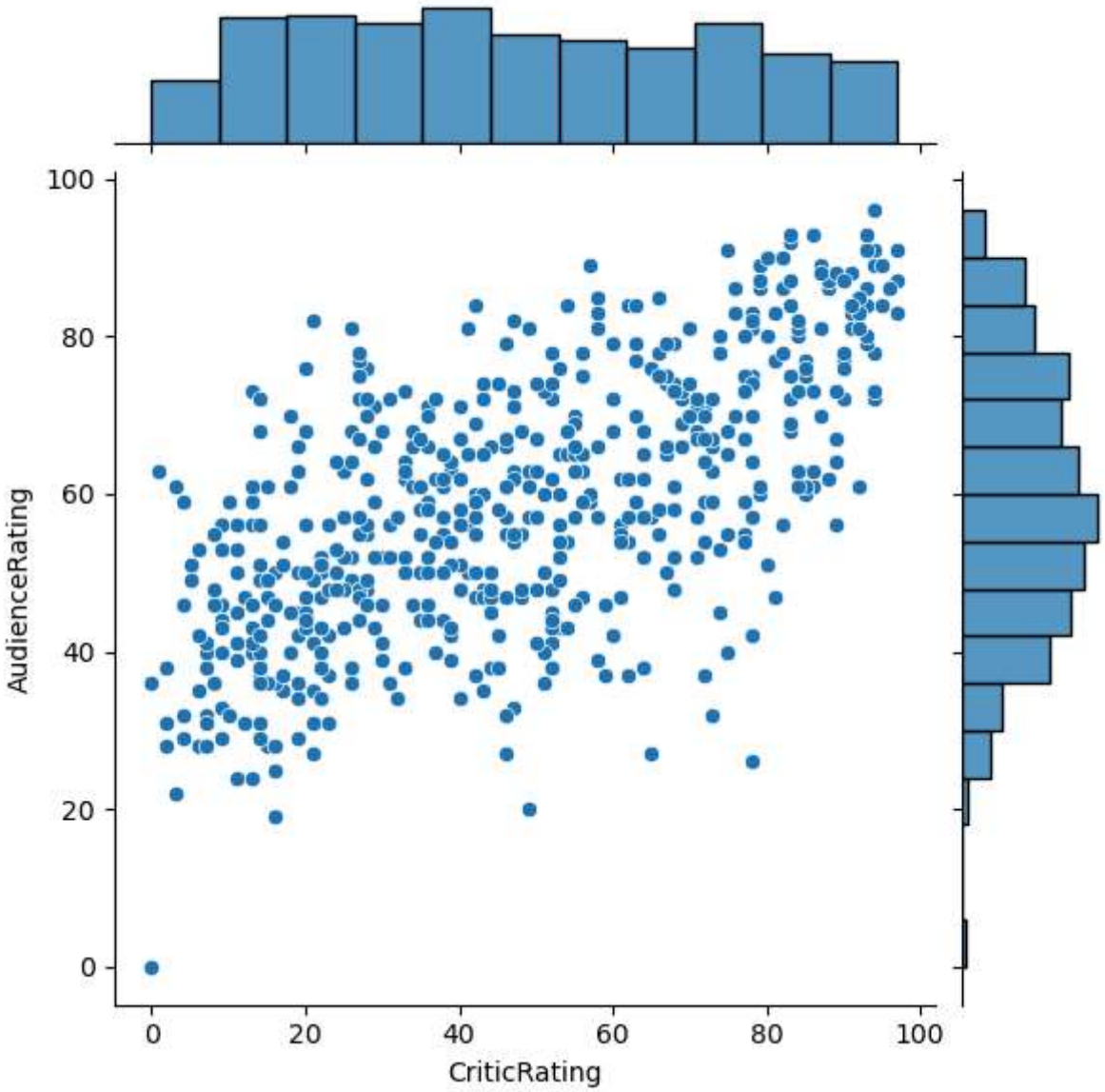
In [25]:
```
j = sns.jointplot(data=movies,x='CriticRating', y= 'AudienceRating')

# plt can't plot this graph cause it's advanced
```

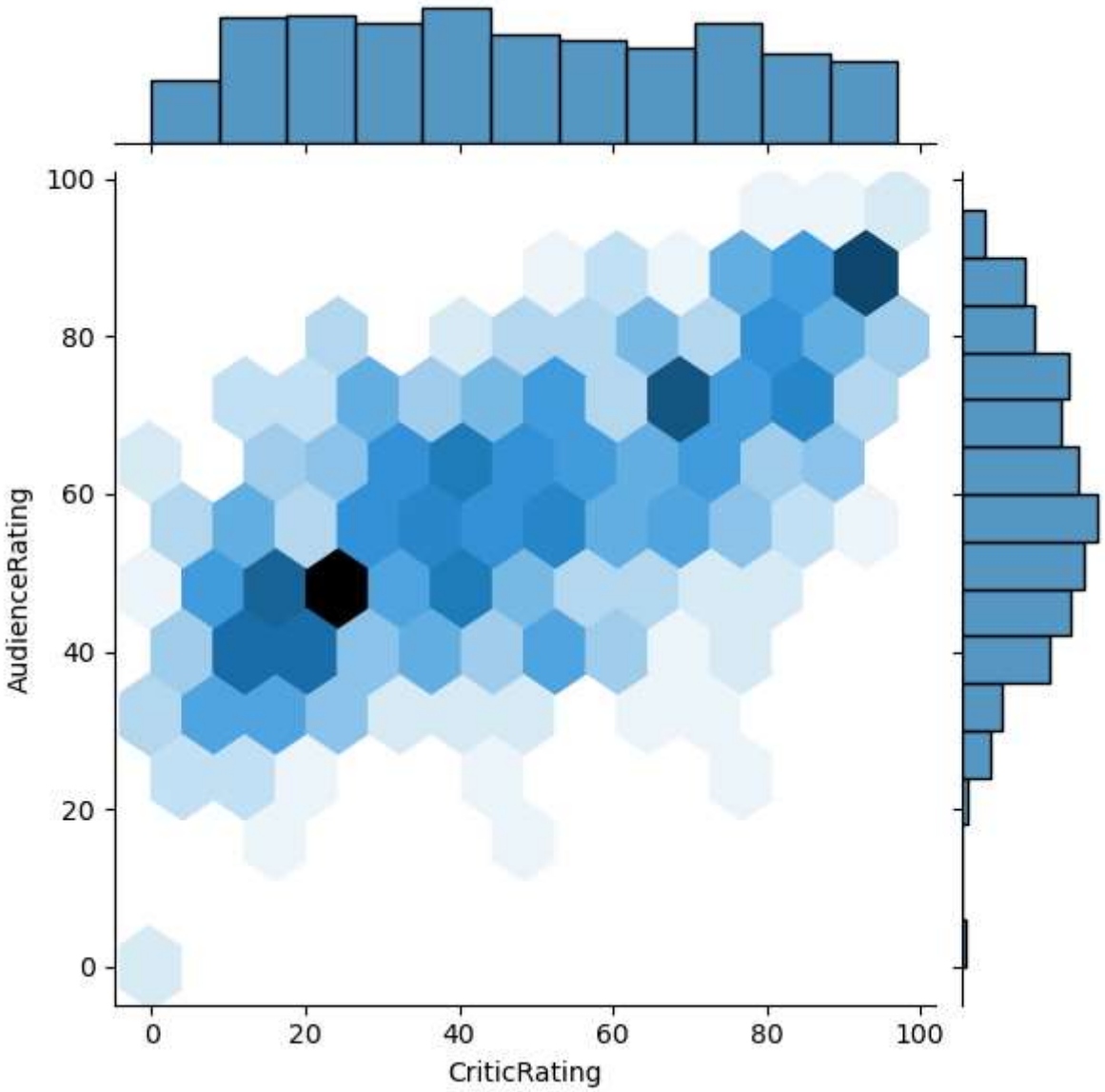
In [26]:
```
j1 = sns.jointplot(data=movies,x='CriticRating', y= 'AudienceRating',kind='hex')

j1.fig
```

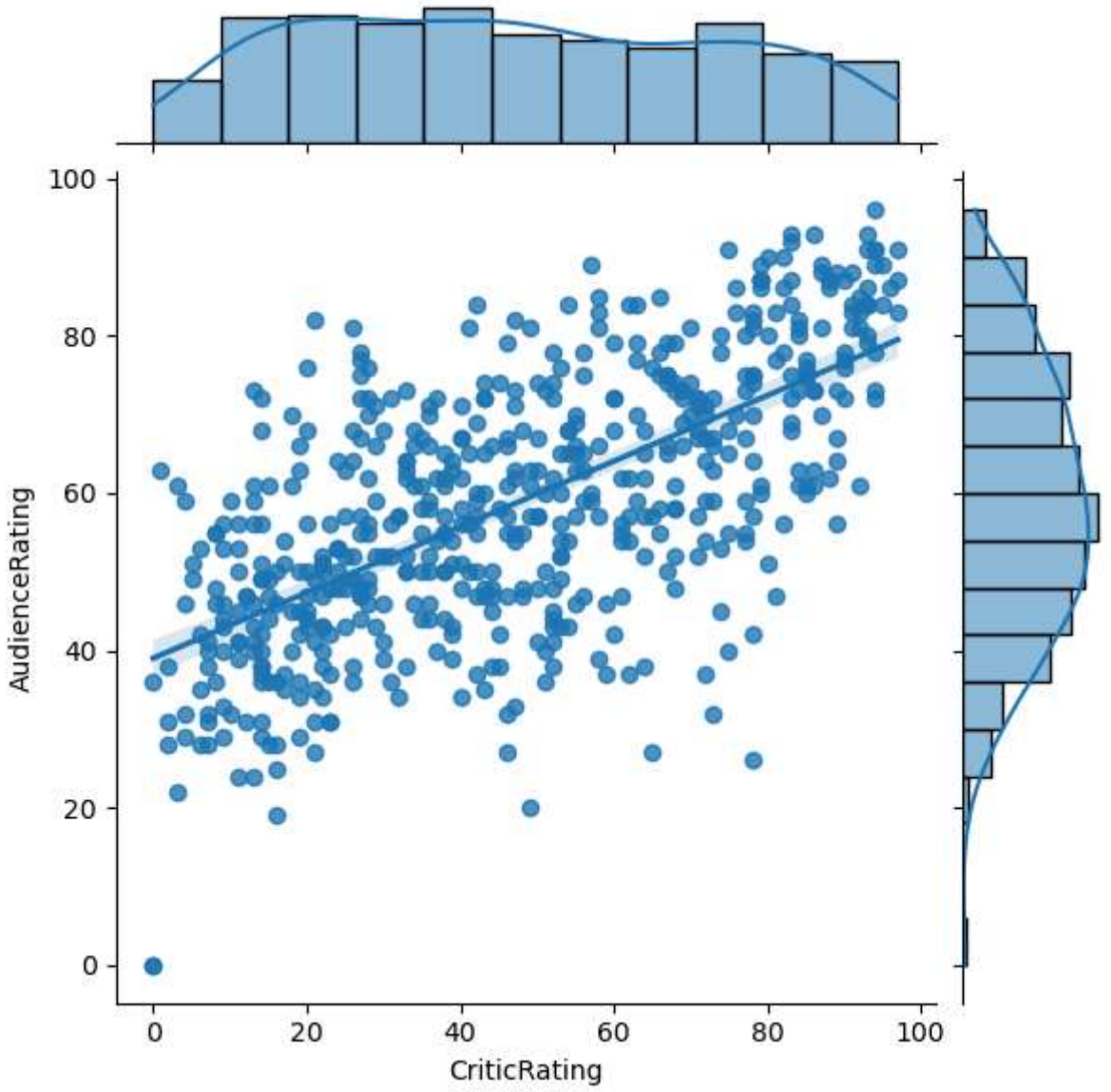
Out[26]:



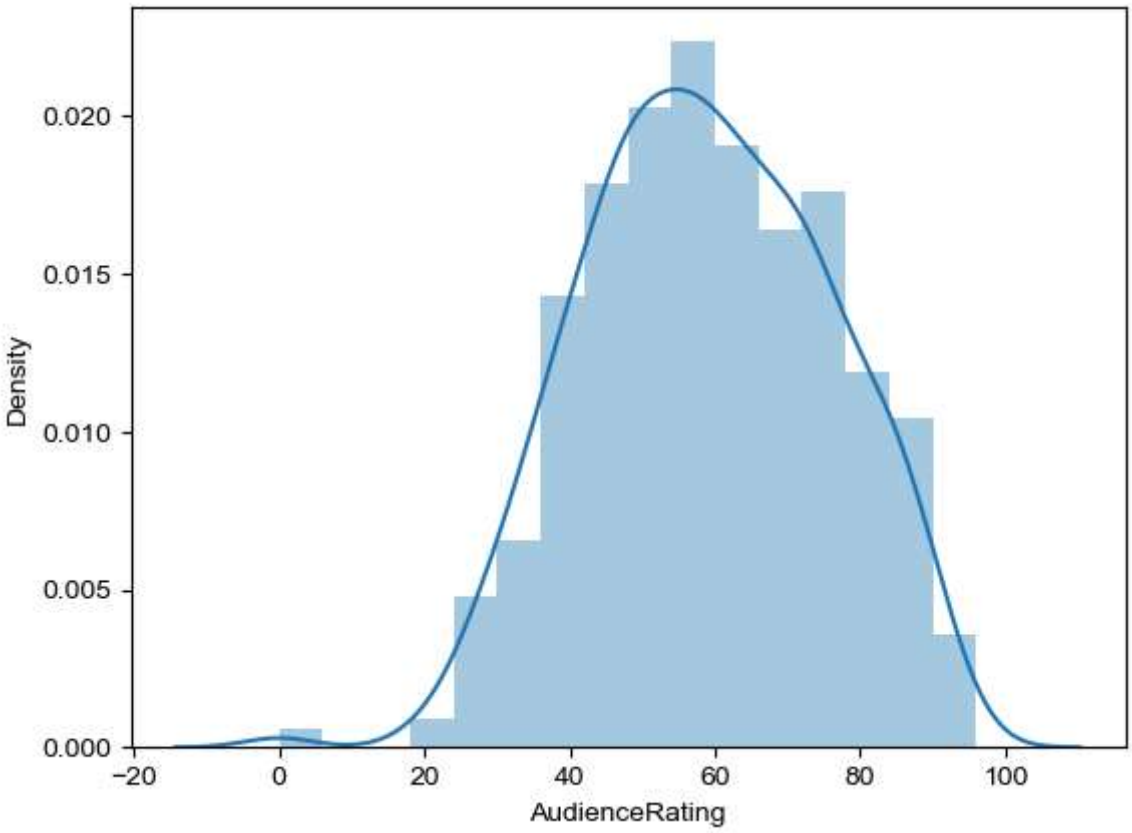
In [27]:
```
j2 = sns.jointplot(data=movies,x='CriticRating', y= 'AudienceRating',kind='reg')
plt.show()
```


100
80
60
40
20
0
AudienceRating
0
20
40
60
80
100
CriticRating

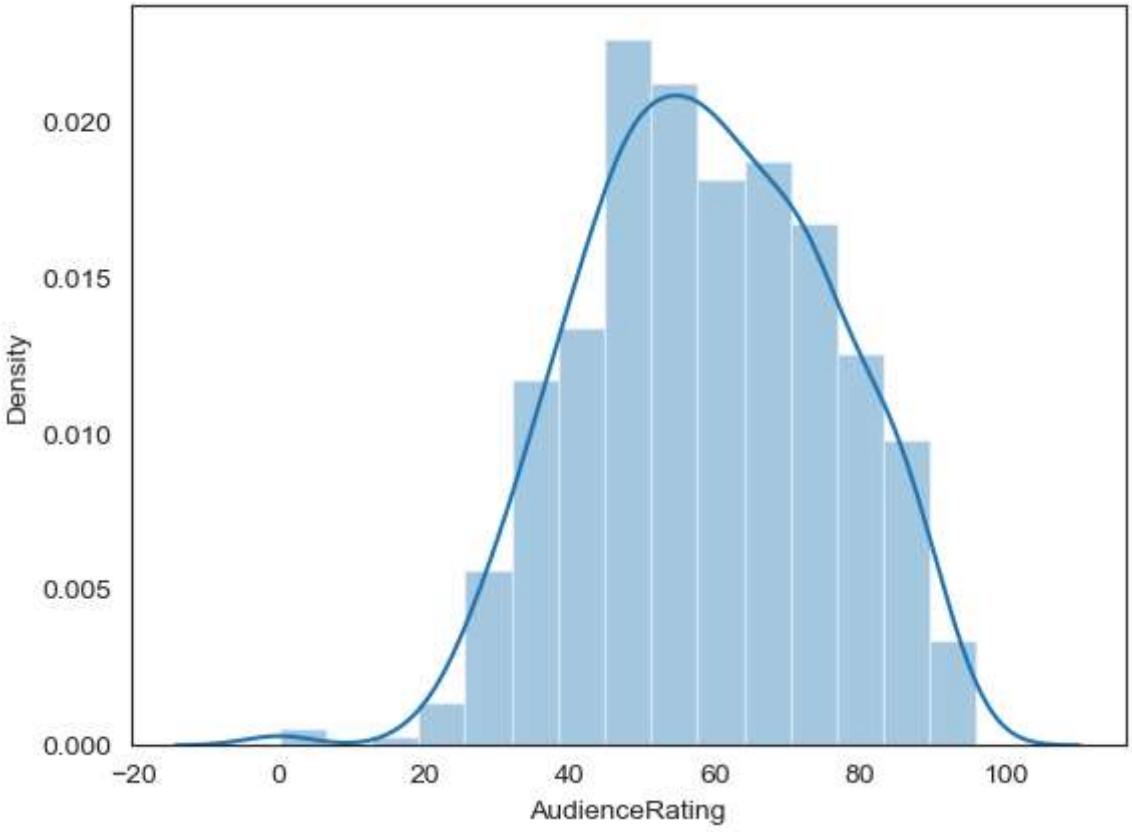

100
80
60
40
20
0
AudienceRating
0
20
40
60
80
100
CriticRating



In [28]:
```
m1 = sns.distplot(movies['AudienceRating'])
sns.set_style('white')    # predefine the look
plt.show()
# univariate analysis
# can also write as m1 = sns.distplot(movies.AudienceRating)
# sns automatically creates y axis
# style-white,dark,whitegrid,darkgrid,ticks
```



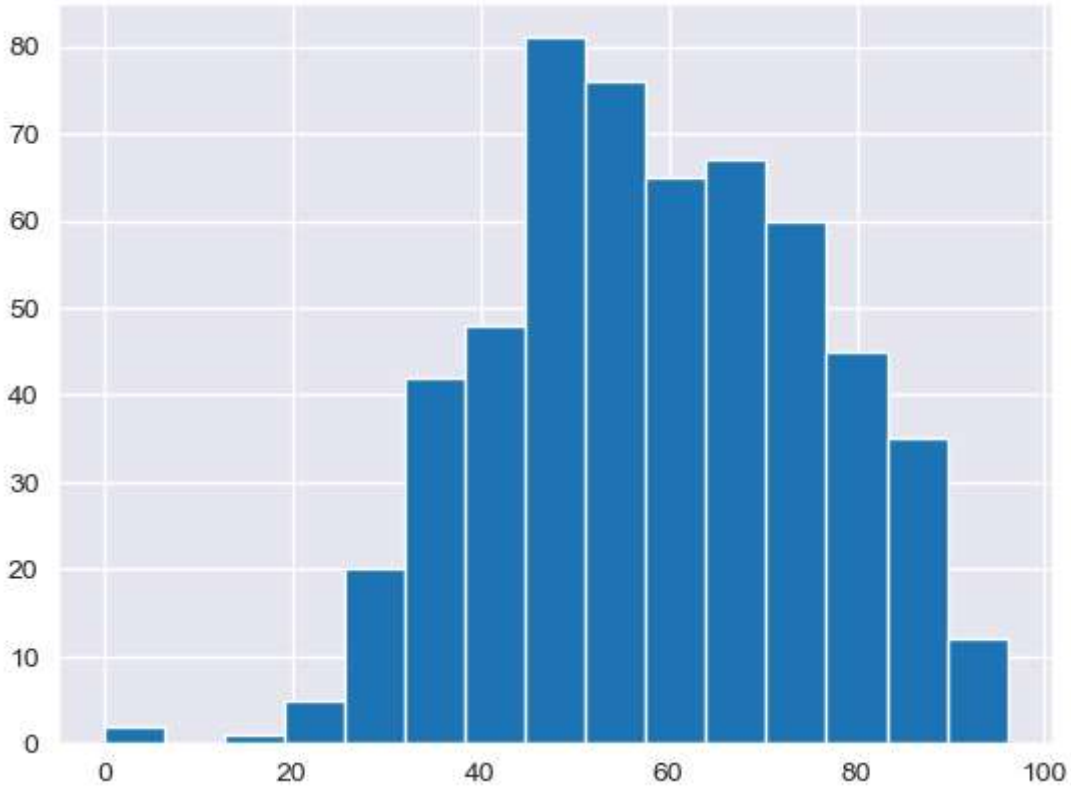
In [29]:
```
m2 = sns.distplot(movies.AudienceRating, bins= 15)
sns.set_style('darkgrid')
plt.show()
```



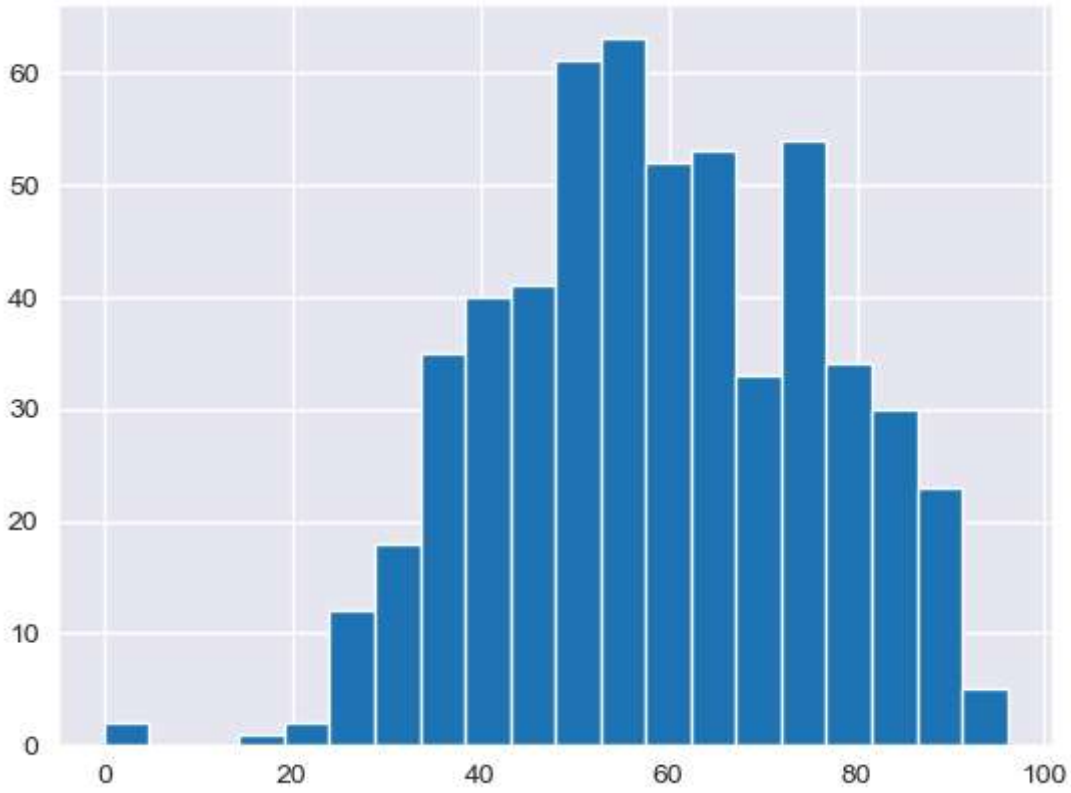
In [30]:
```
n1= sns.hist(movies.AudienceRating, bins= 15)
# sns do not have this attribute
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[30], line 1
----> 1 n1= sns.hist(movies.AudienceRating, bins= 15)

AttributeError: module 'seaborn' has no attribute 'hist'
```

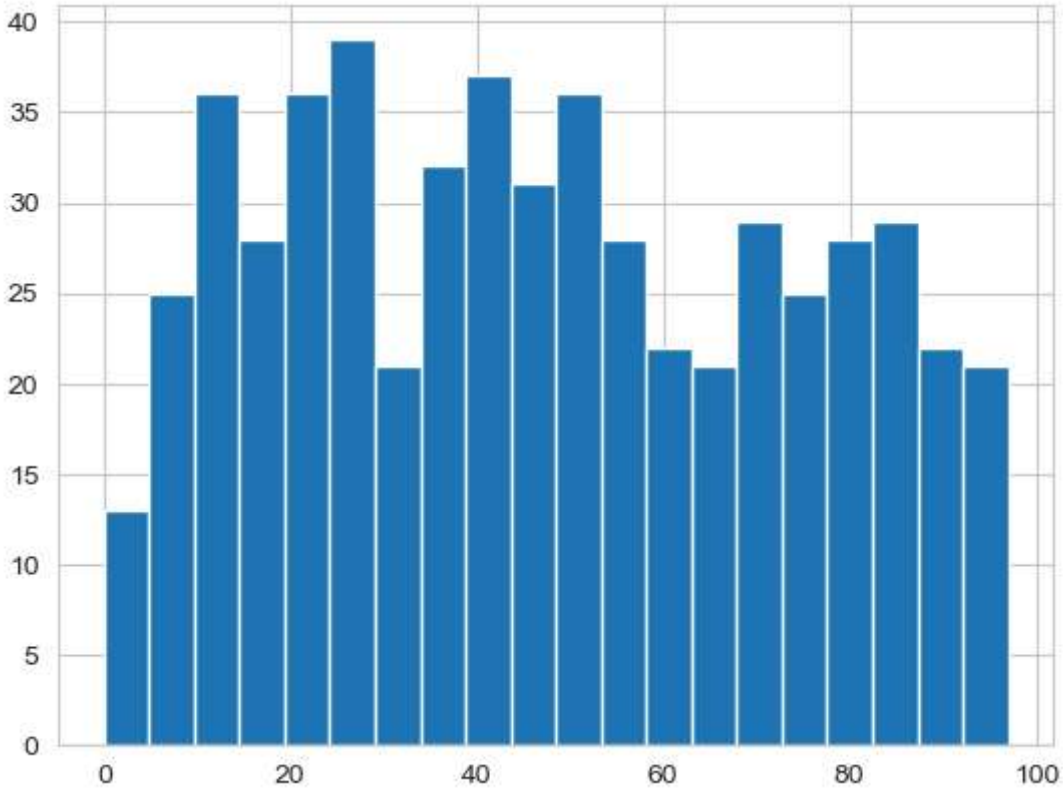
In [31]:
```
n1= plt.hist(movies.AudienceRating, bins= 15)
plt.show()
```



In [32]:
```
n2= plt.hist(movies.AudienceRating, bins= 20)
sns.set_style('whitegrid')
plt.show()
```

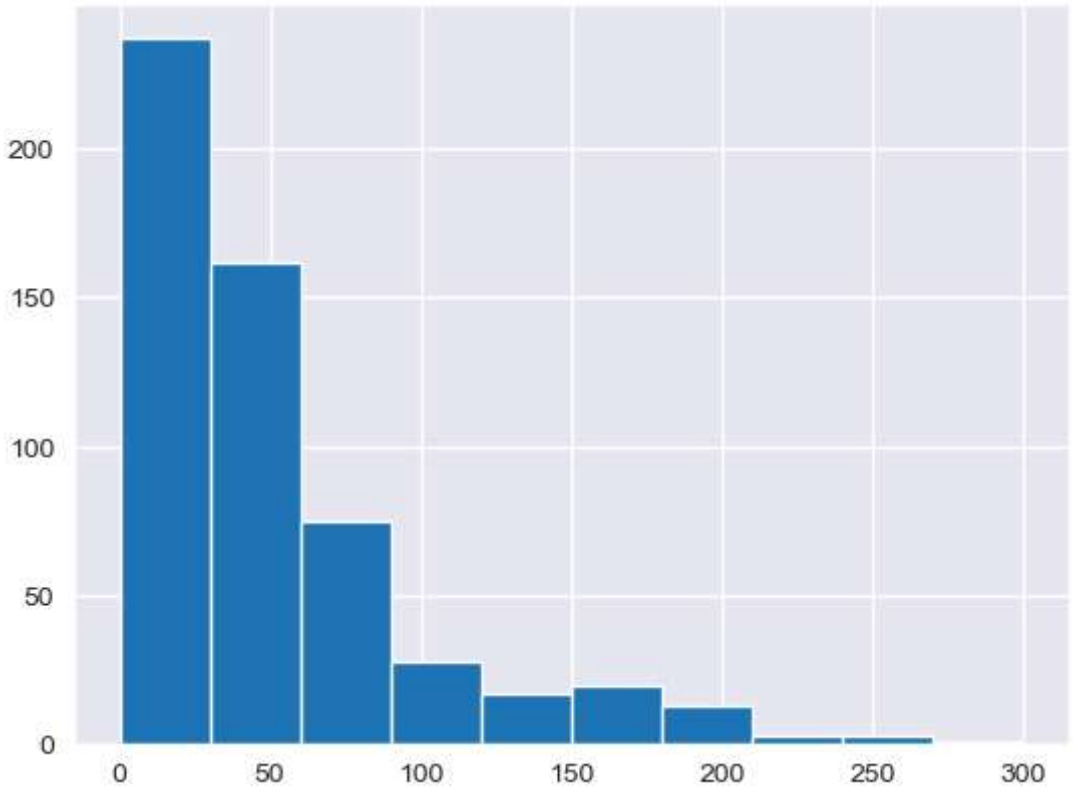


In [33]:
```
n3= plt.hist(movies.CriticRating, bins= 20)
sns.set_style('darkgrid')
plt.show()
```

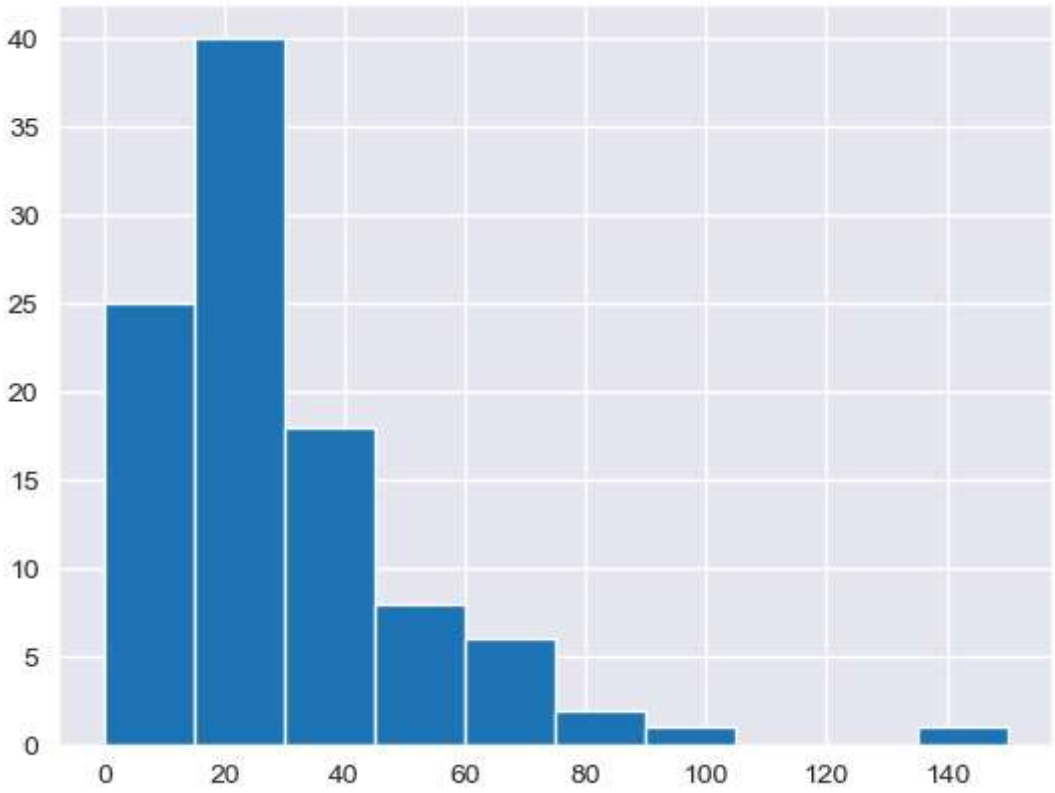


In [34]:
```
# creating stacked histograms
```

```
h1= plt.hist(movies.BudgetMillions)
plt.show()
```



In [35]:
```
# plot histogram of budgetmillion attribute for the drama genre movies

h2 = plt.hist(movies[movies.Genre== 'Drama'].BudgetMillions)
plt.show()
```



In [36]:
```
movies.head()
```

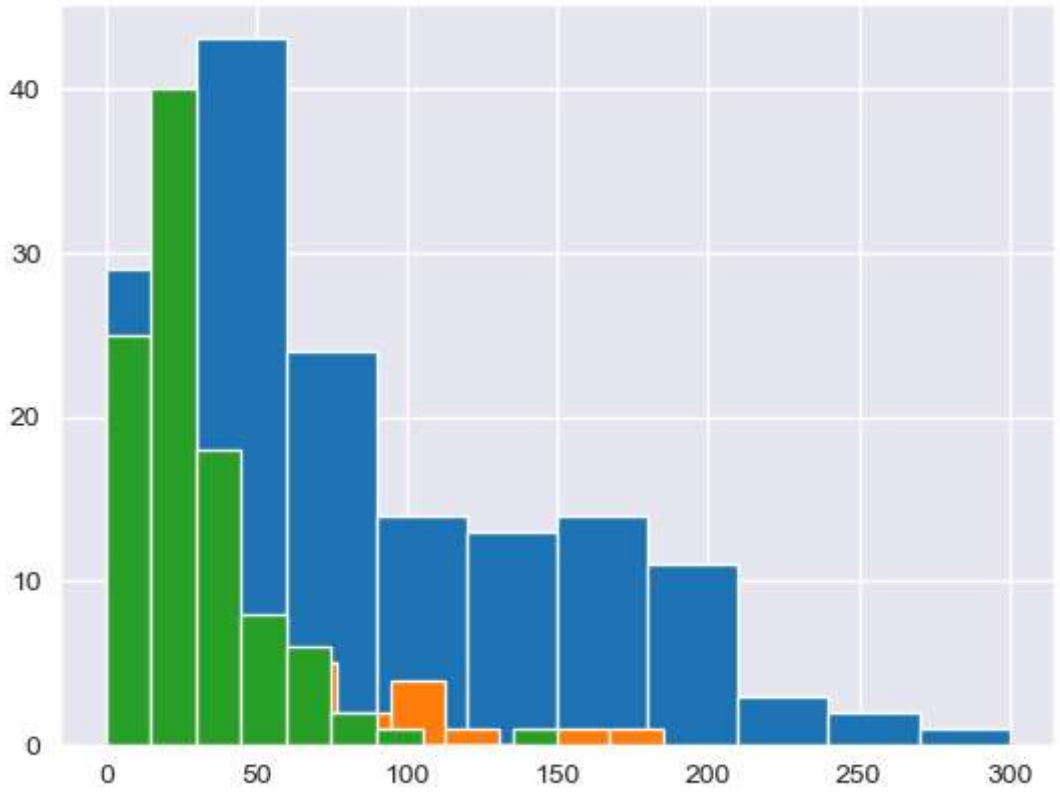
Out[36]:

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

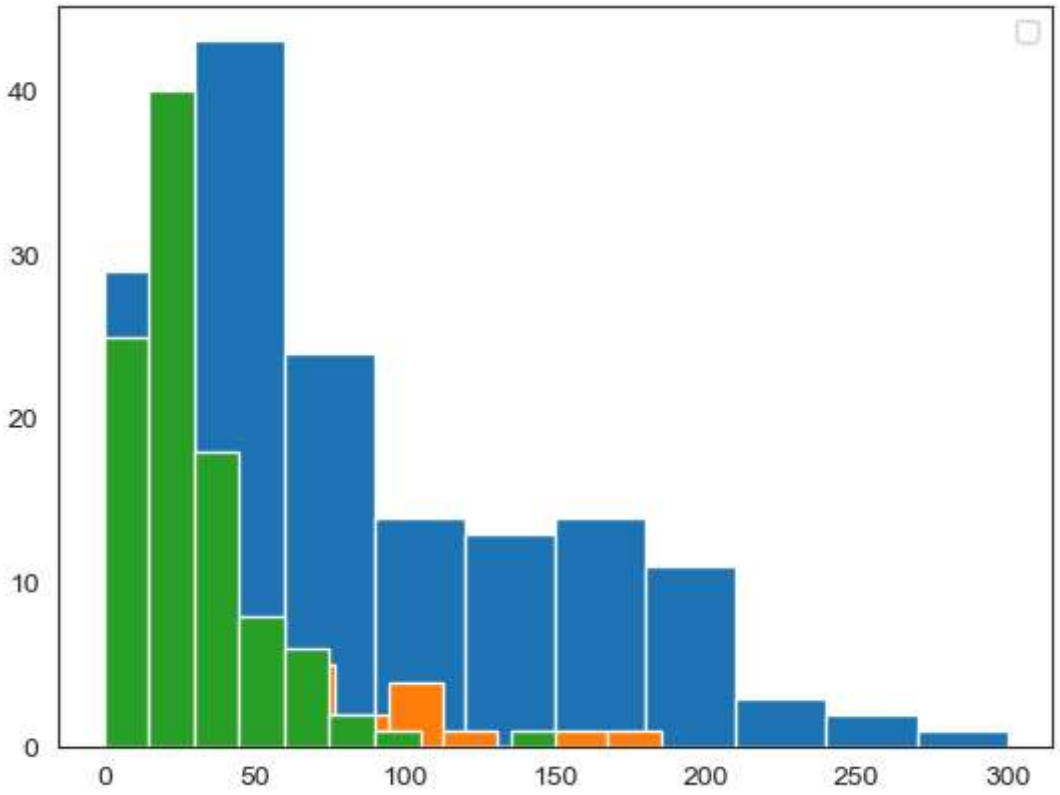
In [37]:
```
movies.Genre.unique()
```

Out[37]:
```
['Comedy', 'Adventure', 'Action', 'Horror', 'Drama', 'Romance', 'Thriller']
Categories (7, object): ['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Roma
nce', 'Thriller']
```

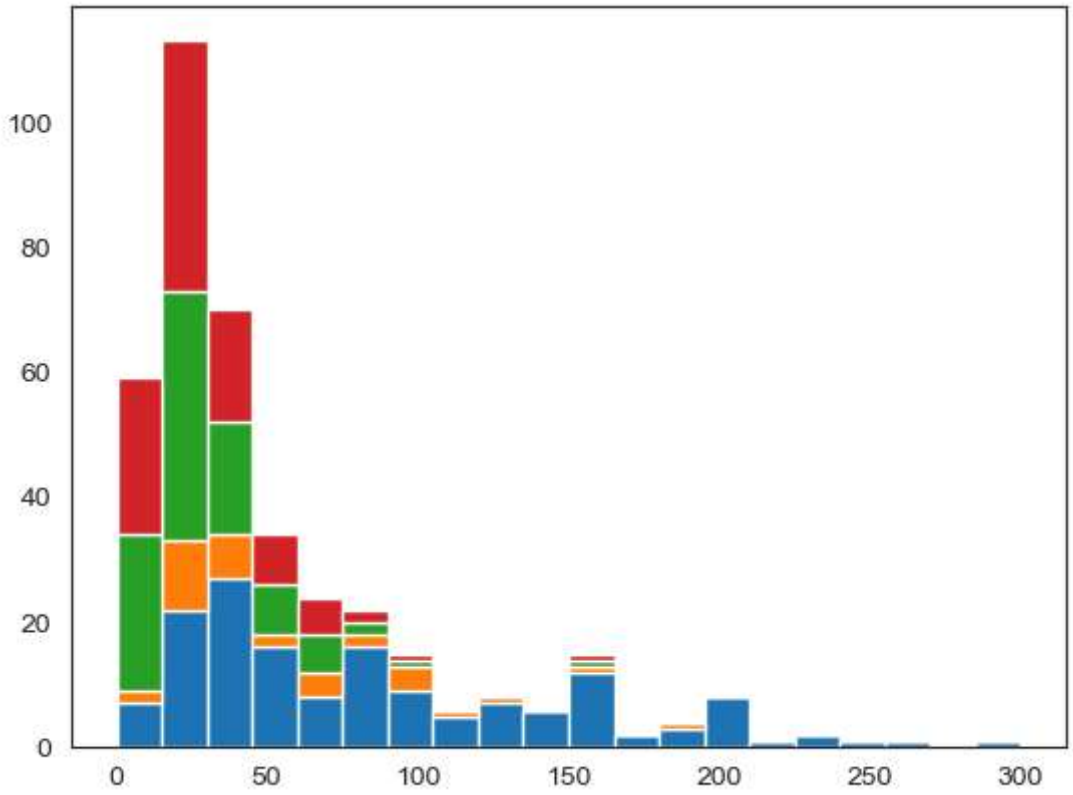
In [38]:
```
# Below plots are stacked histogram becuase overlaped

plt.hist(movies[movies.Genre== 'Action'].BudgetMillions)
plt.hist(movies[movies.Genre== 'Thriller'].BudgetMillions)
plt.hist(movies[movies.Genre== 'Drama'].BudgetMillions)
sns.set_style('white')
plt.show()
```



In [39]:
```
plt.hist(movies[movies.Genre== 'Action'].BudgetMillions)
plt.hist(movies[movies.Genre== 'Thriller'].BudgetMillions)
plt.hist(movies[movies.Genre== 'Drama'].BudgetMillions)
sns.set_style('white')
plt.legend()
plt.show()

# No handles with labels found to put in legend.
```



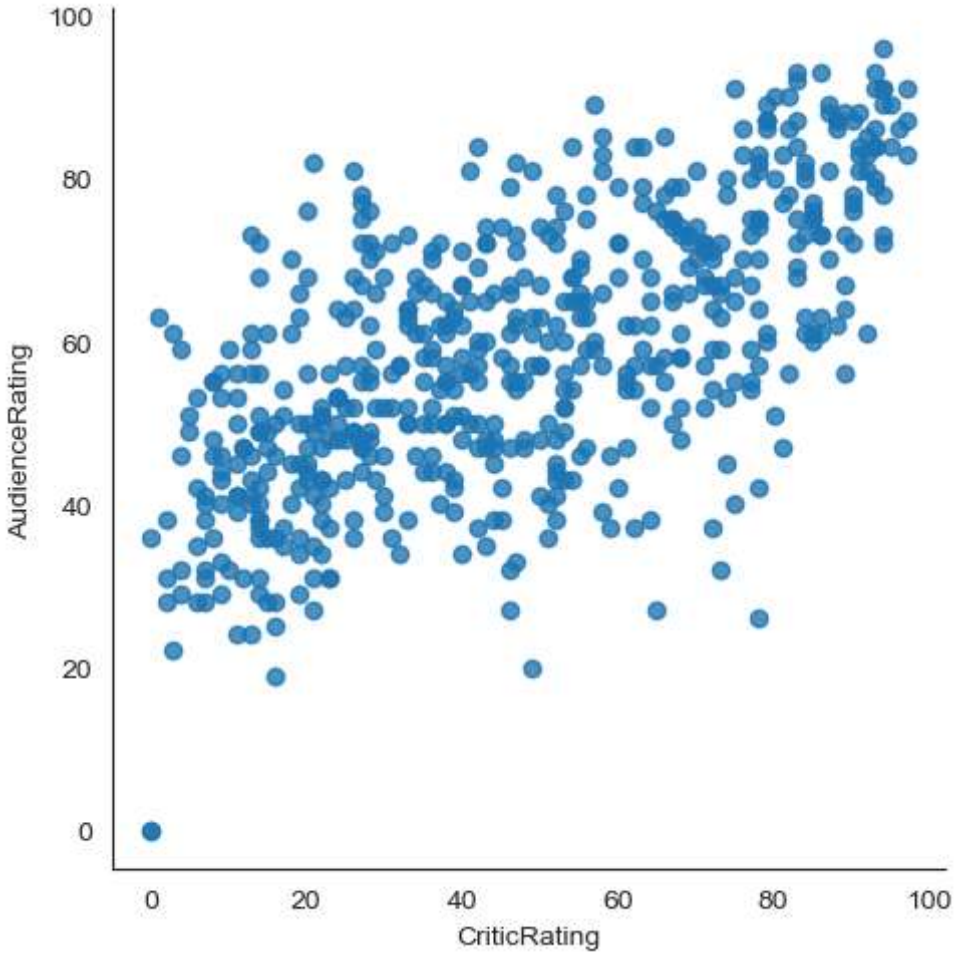
In [40]:
```
plt.hist([movies[movies.Genre== 'Action'].BudgetMillions,\
          movies[movies.Genre== 'Thriller'].BudgetMillions,\
          movies[movies.Genre== 'Drama'].BudgetMillions,\
          movies[movies.Genre== 'Drama'].BudgetMillions],
         bins=20, stacked = True)

plt.show()
```



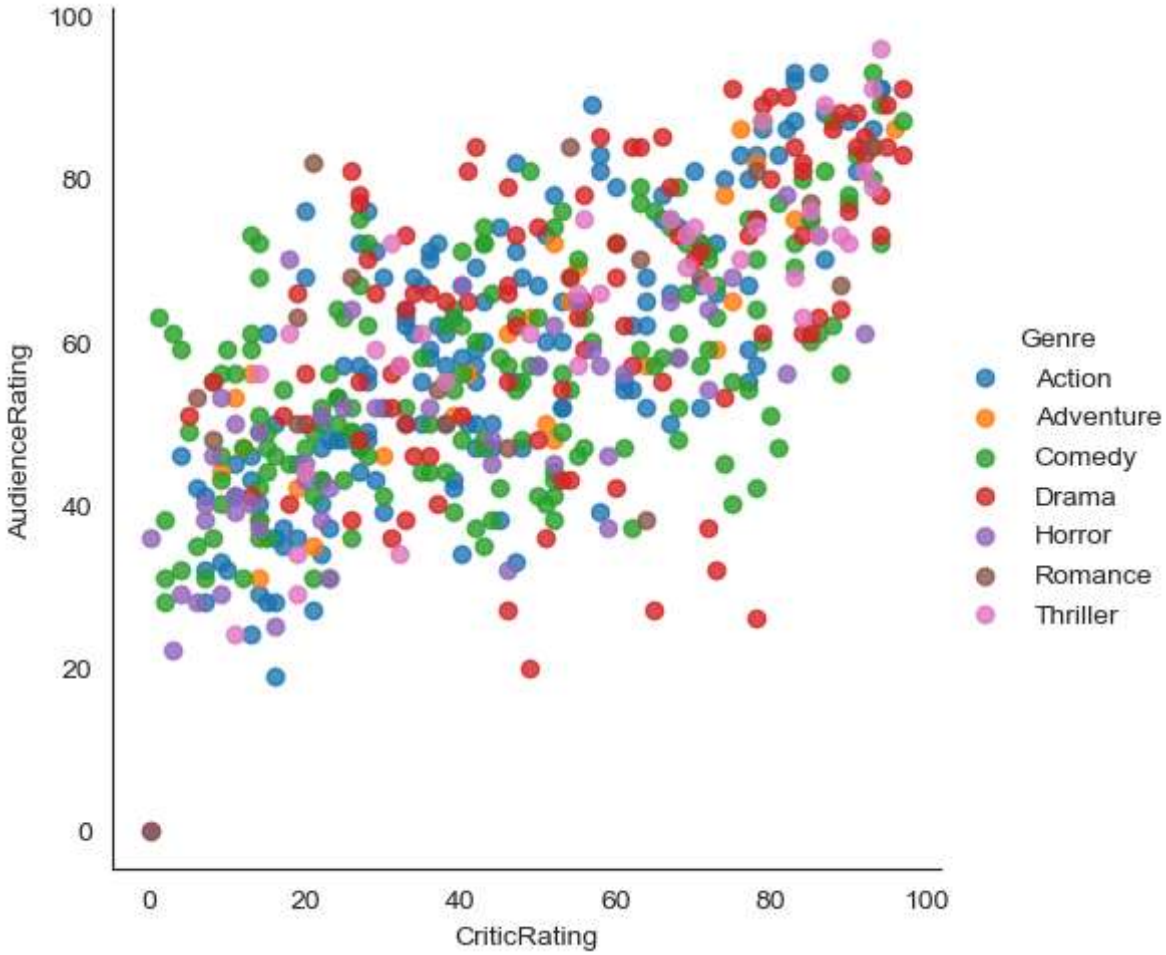
In [41]:
```
# if you have 100 or more such categories you cannot copy & paste all the things

for gen in movies.Genre.cat.categories:    # acces only unique values
    print(gen)
```

```
Action
Adventure
Comedy
Drama
Horror
Romance
Thriller
```

In [42]:
```
vis1 = sns.lmplot(data=movies, x= 'CriticRating', y= 'AudienceRating',\
                  fit_reg=False)
plt.show()
```



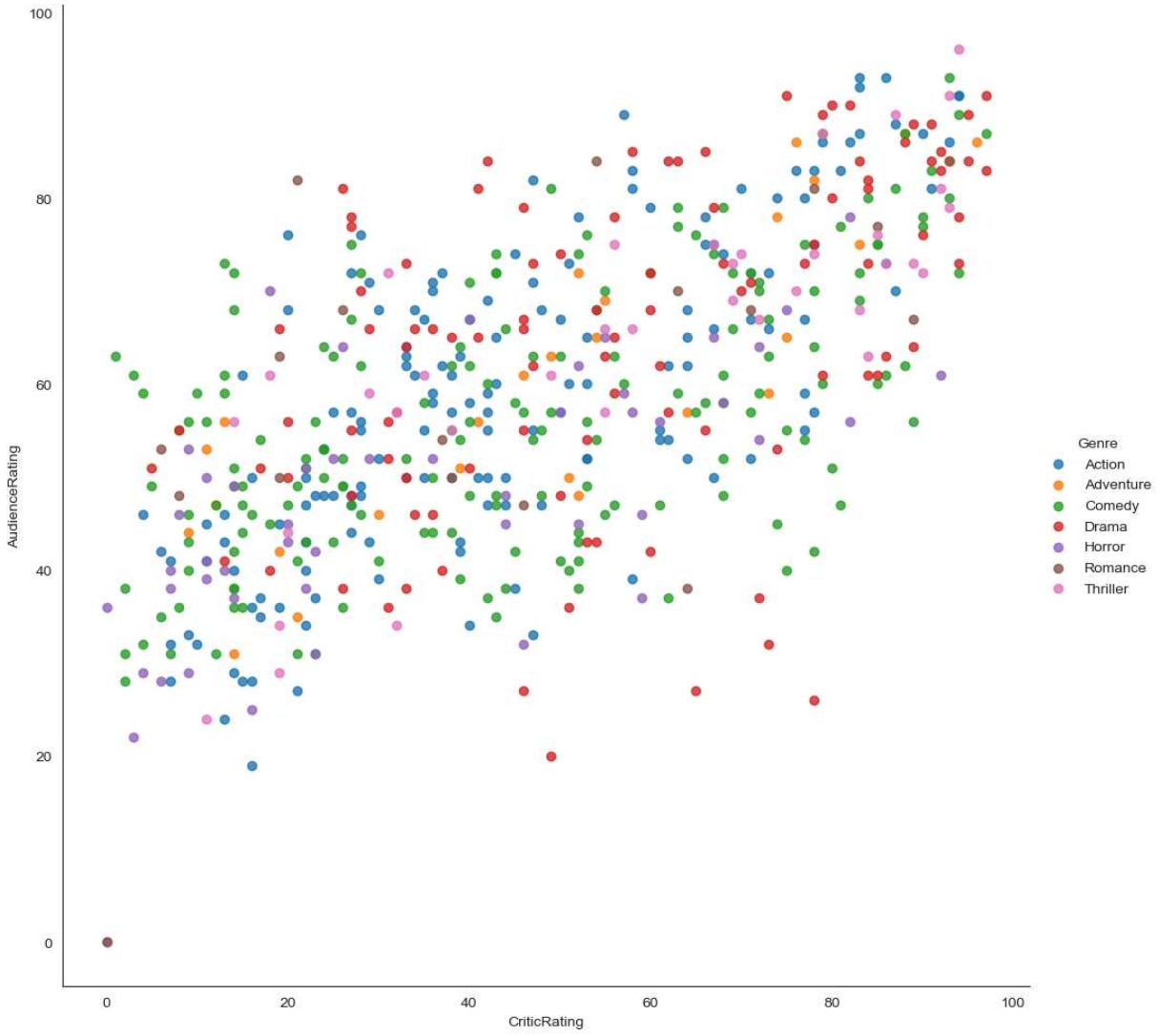
In [43]:
```
vis1 = sns.lmplot(data=movies, x= 'CriticRating', y= 'AudienceRating',\
                  fit_reg=False, hue='Genre')
plt.show()
```



In [44]:
```
vis1 = sns.lmplot(data=movies, x= 'CriticRating', y= 'AudienceRating',\
                  fit_reg=False, hue='Genre',size=10)
plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[44], line 1
----> 1 vis1 = sns.lmplot(data=movies, x= 'CriticRating', y= 'AudienceRating',\
      2                   fit_reg=False, hue='Genre',size=10)
      3 plt.show()

TypeError: lmplot() got an unexpected keyword argument 'size'
```

In [50]:
```
vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\
                  fit_reg=False, hue = 'Genre', height = 10,aspect=1)
plt.show()

# in new version of seaborn for lmplot, argument size is deprecated and replaced by
```



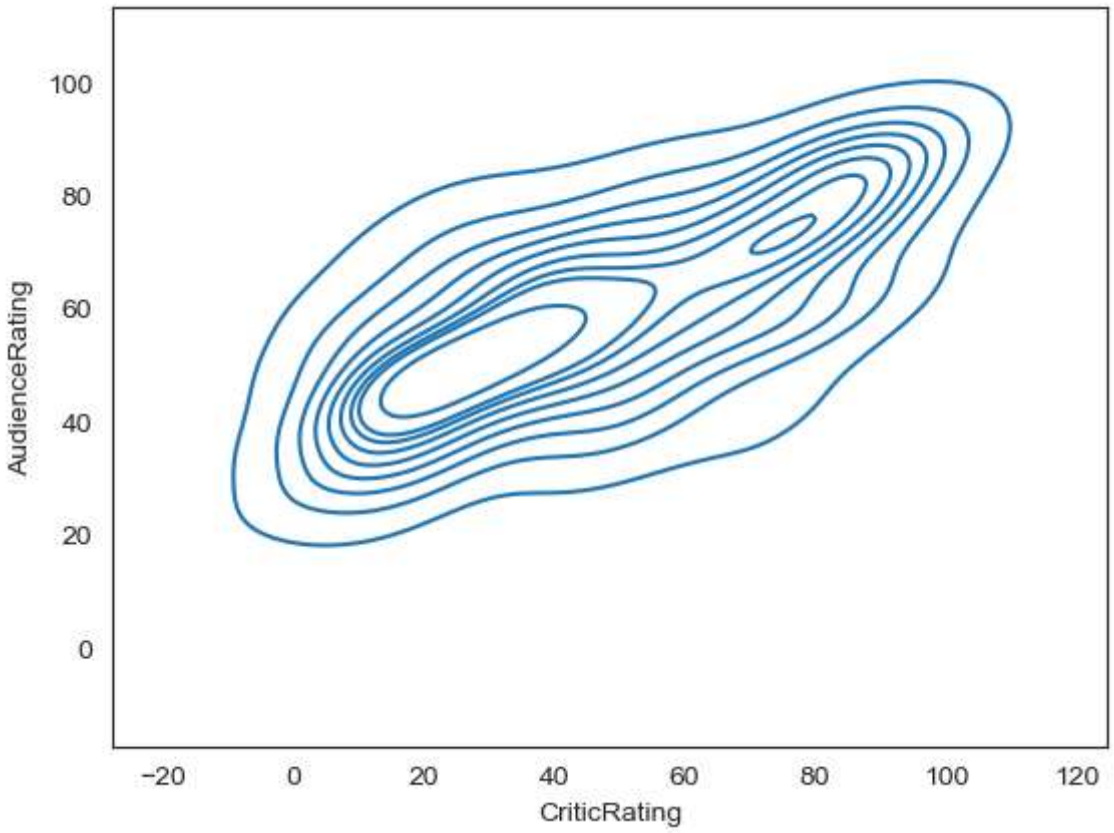
In [51]:
```
# Kernal Density Estimate plot ( KDE PLOT)
k1= sns.kdeplot(movies.CriticRating,movies.AudienceRating)

#center point is kernal ; it is called kde ; instead of dot it visualize like this
# spread at the audience ratings can clearly seen
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[51], line 2
      1 # Kernal Density Estimate plot ( KDE PLOT)
----> 2 k1= sns.kdeplot(movies.CriticRating,movies.AudienceRating)

TypeError: kdeplot() takes from 0 to 1 positional arguments but 2 were given
```

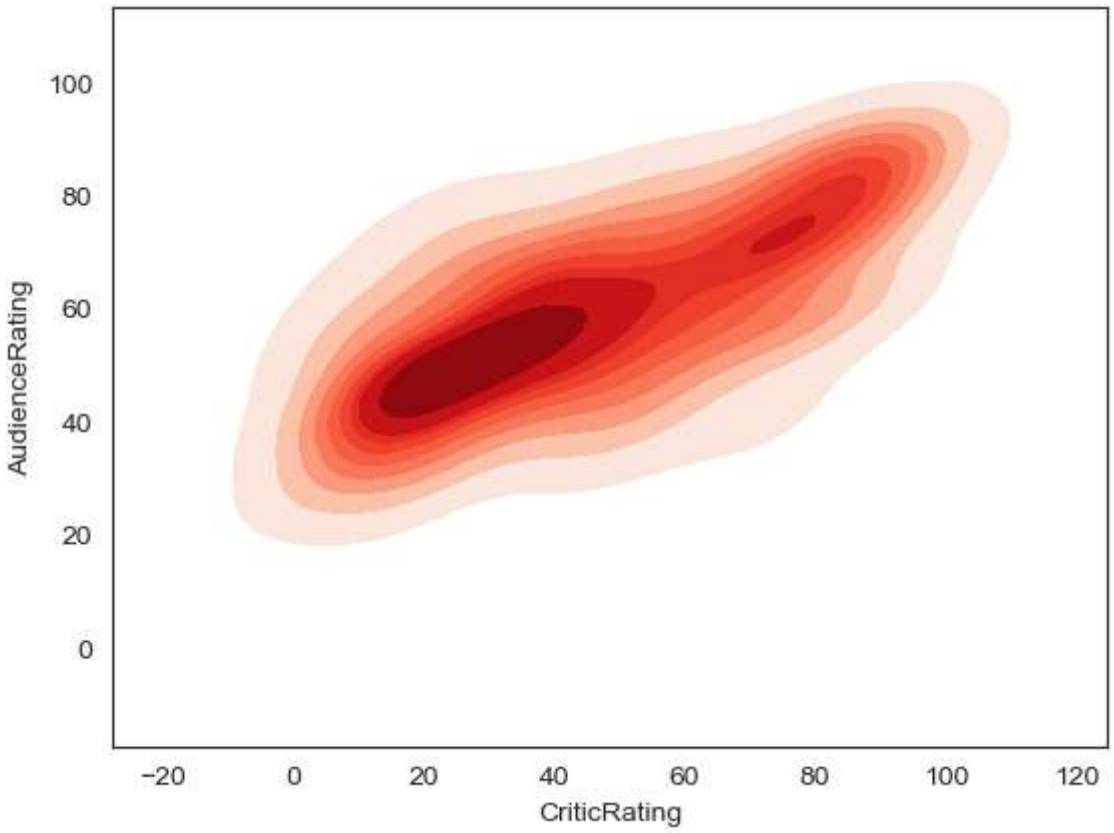
In [68]:
```
sns.__version__
```

Out[68]: '0.13.2'

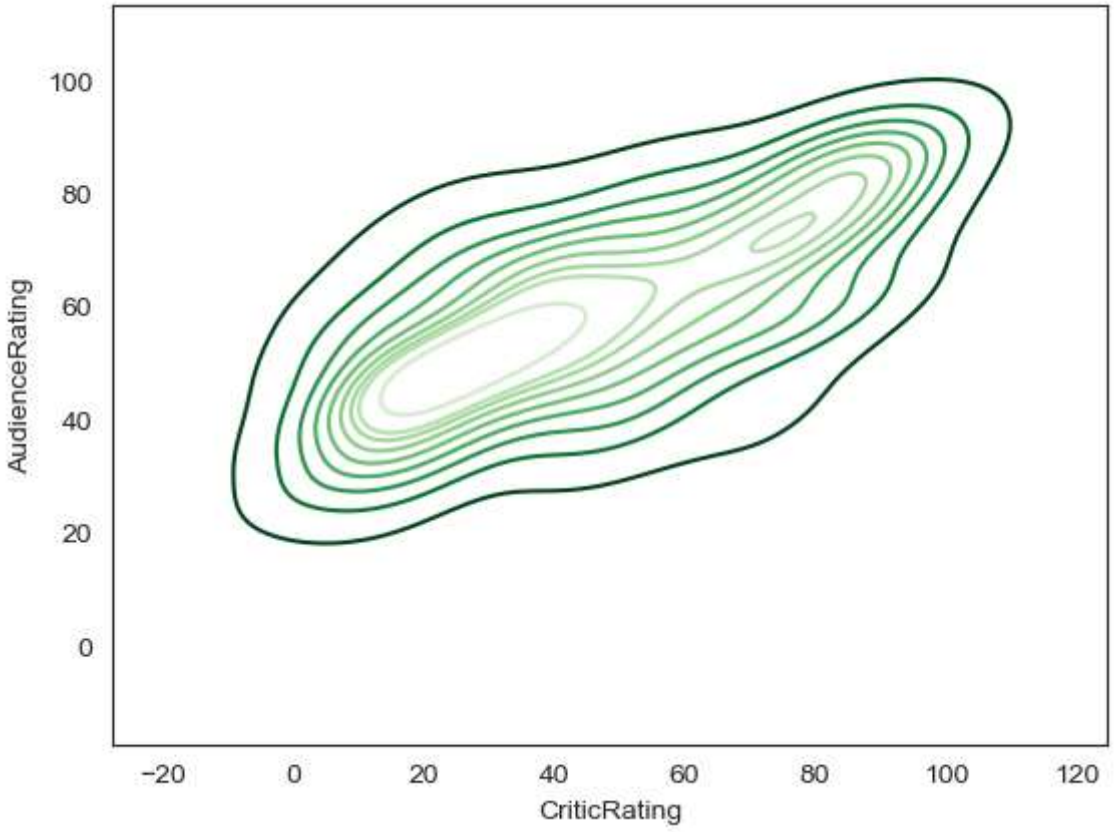
In [69]:
```
k1 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating)
plt.show()
# as we are using version 0.13.2 of seaborn we need to write keyword in argument
```



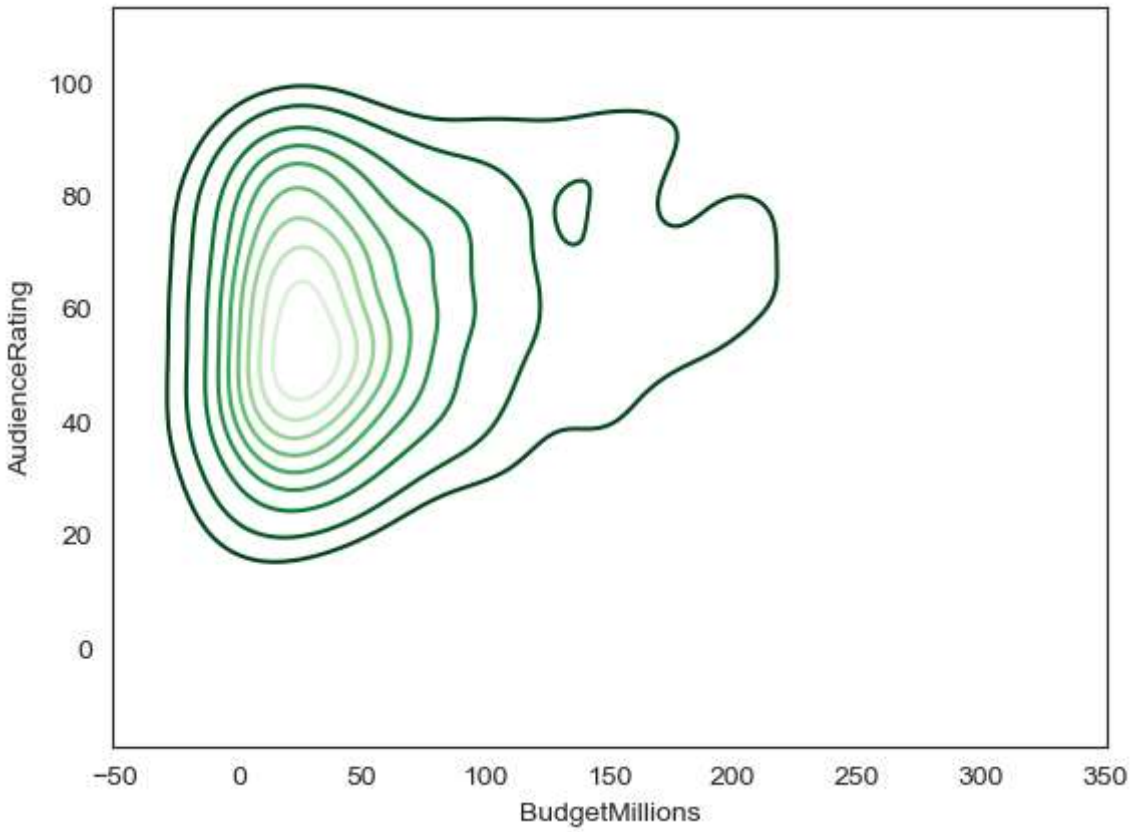
In [70]:
```
k1 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, shade=True,shade_l
plt.show()

#shade-area under kde; shade_lowest- background/area other than kde;cmap-color to k
```



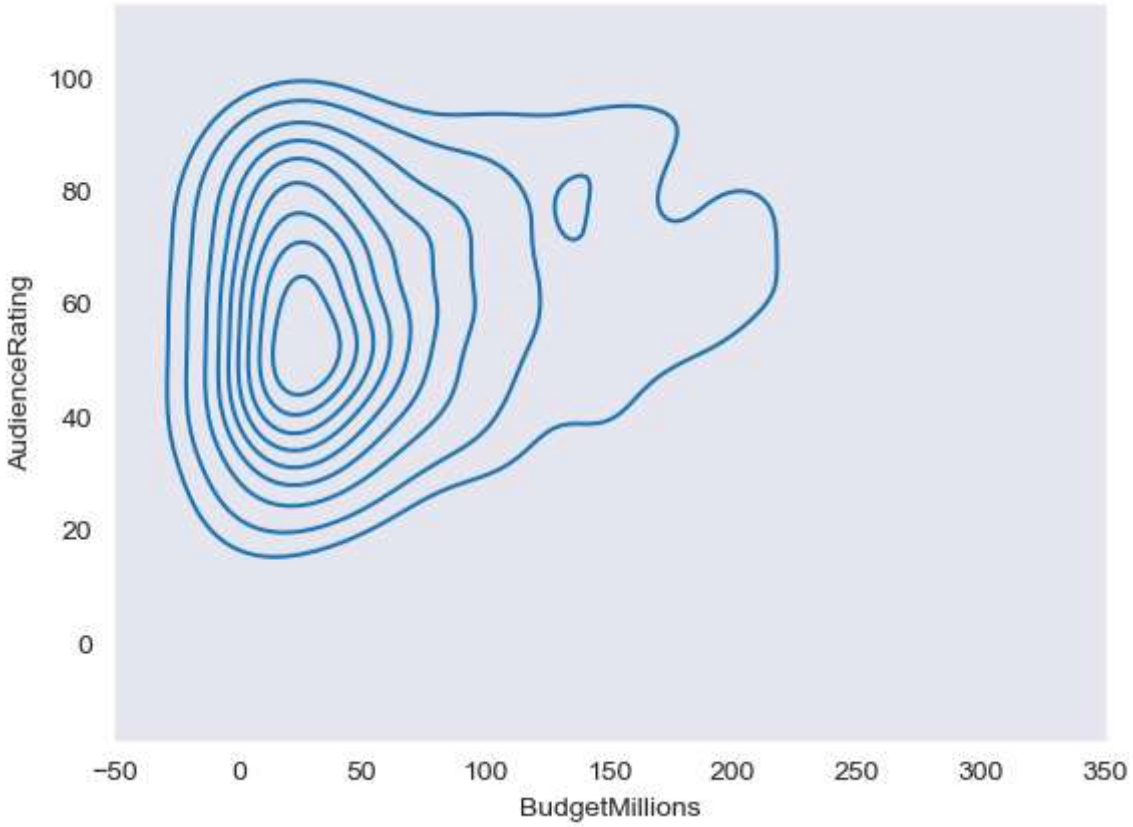
In [71]:
```
k1 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating,shade_lowest=False,
plt.show()
```



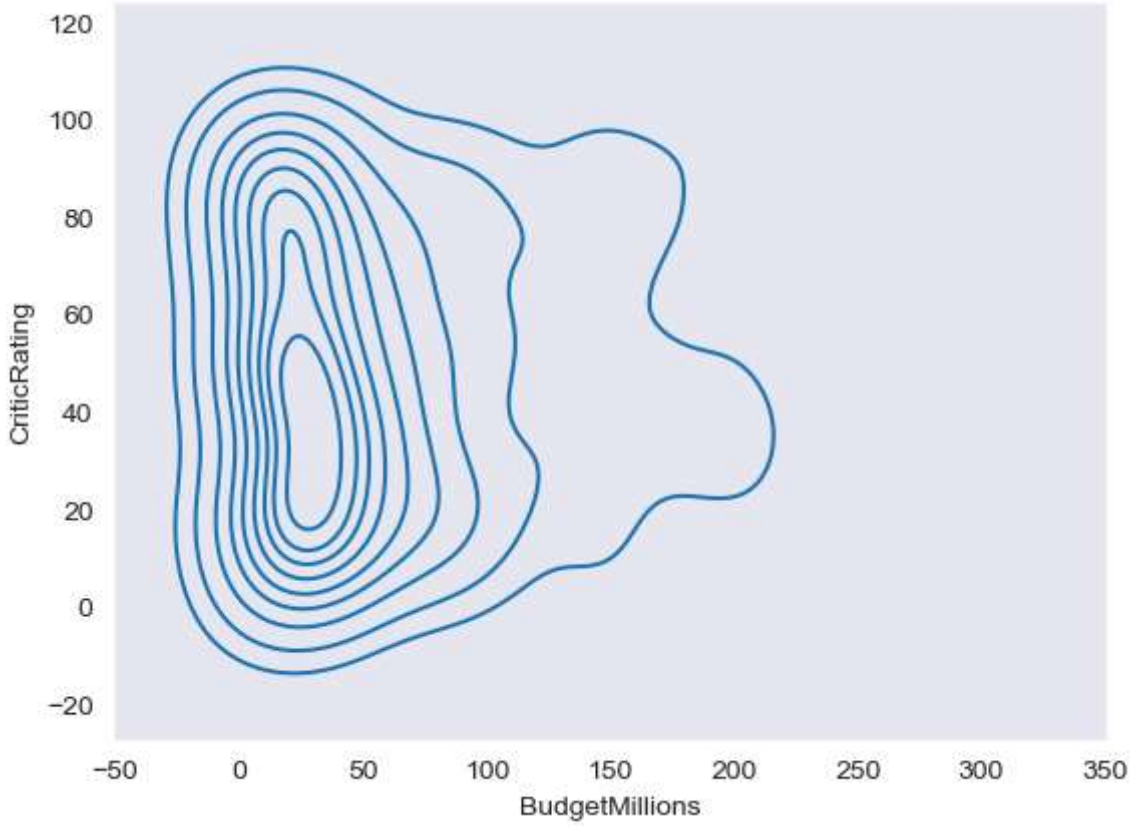
In [72]:
```
k5 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating,shade_lowest=Fals
plt.show()
```



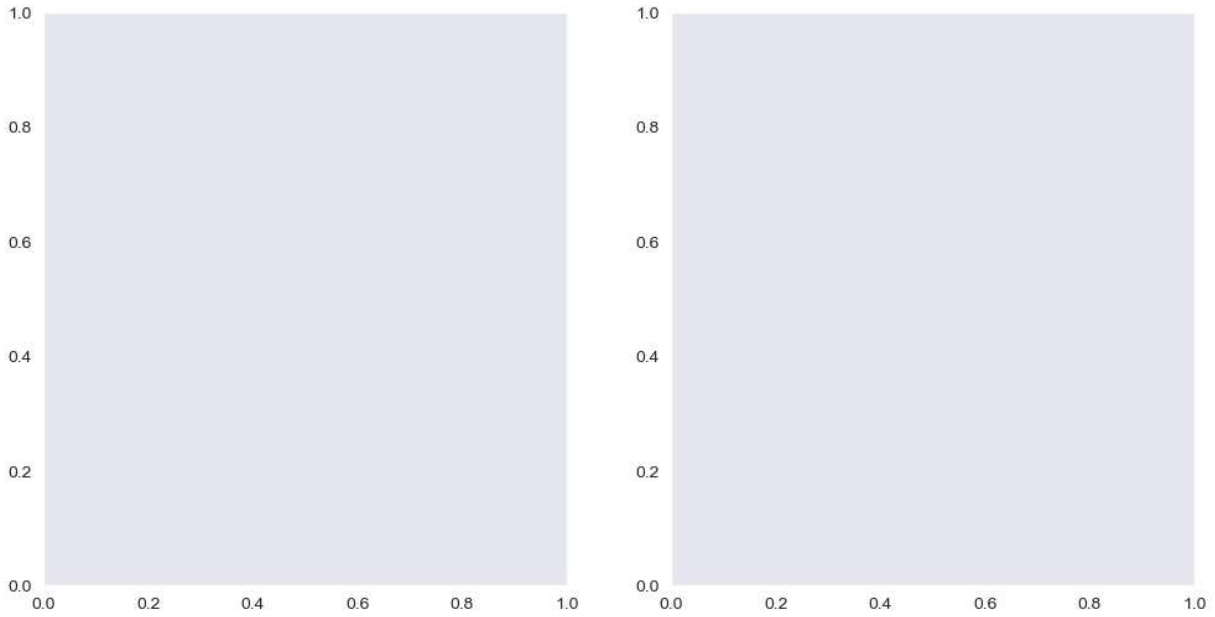
In [73]:
```
sns.set_style('dark')
k6 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating)
plt.show()
```



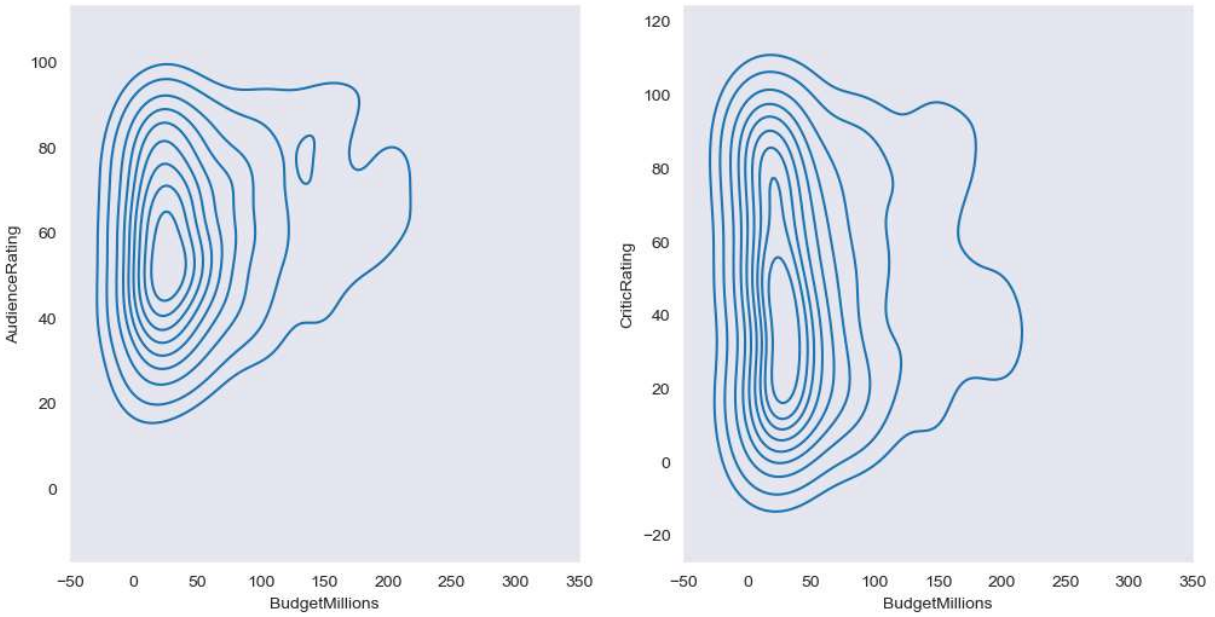
In [74]:
```
k7 = sns.kdeplot(x=movies.BudgetMillions, y=movies.CriticRating)
plt.show()
```



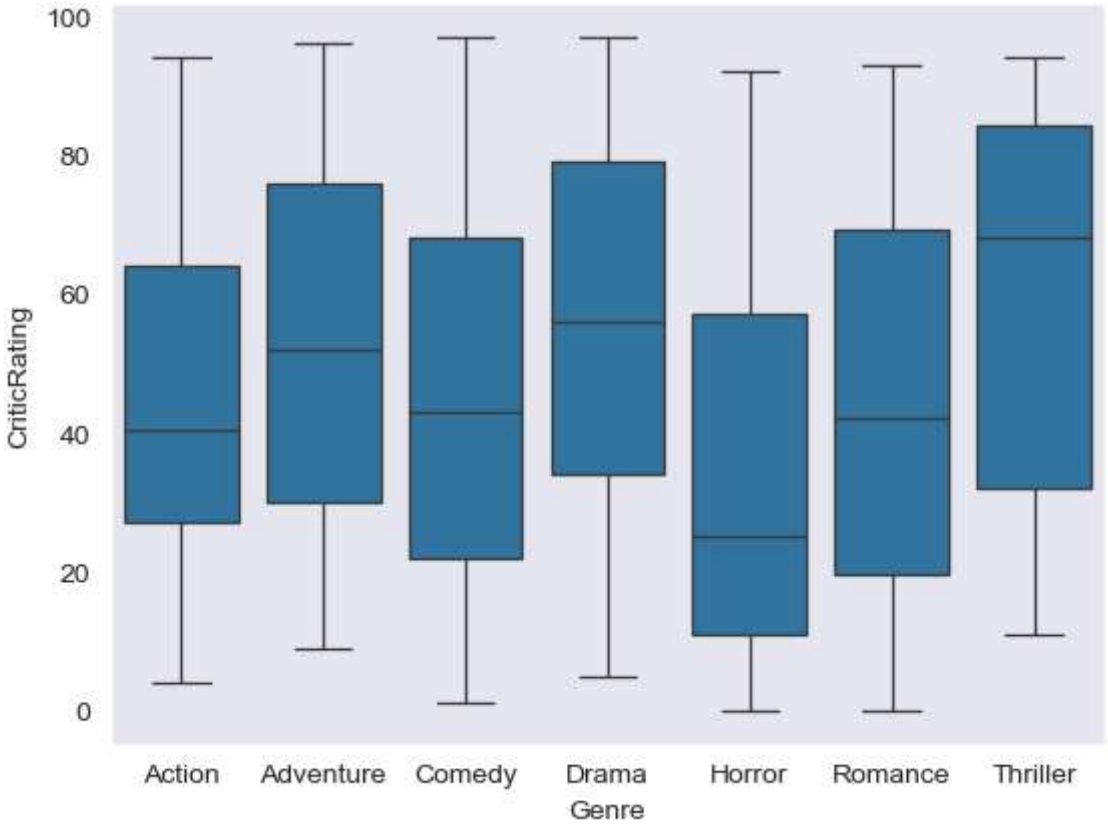
In [75]:
```
f, axes = plt.subplots(1,2,figsize = (12,6))   # 1 row and 2 col; figsize 12x6
plt.show()
```



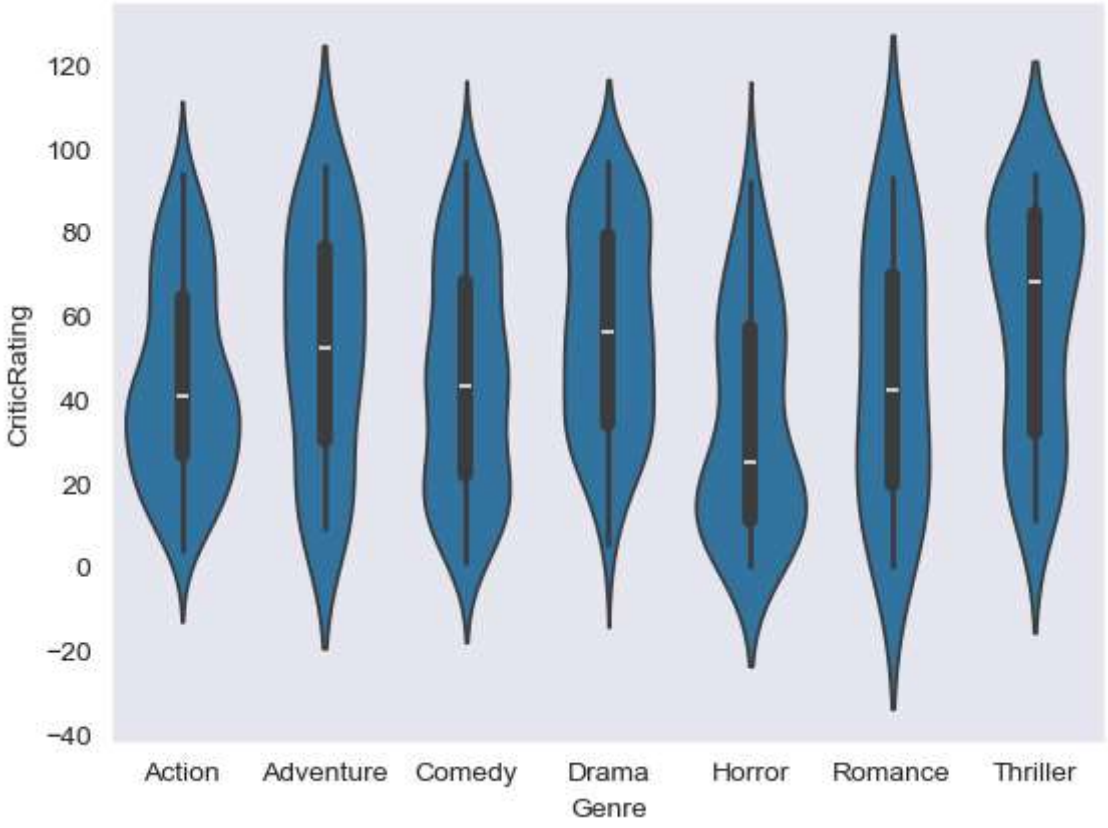
In [76]:
```
# subplot

f, axes = plt.subplots(1,2,figsize = (12,6))   # 1 row and 2 col; figsize 12x6

k1 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating, ax=axes[0])

k2 = sns.kdeplot(x=movies.BudgetMillions, y=movies.CriticRating, ax=axes[1])

plt.show()
```



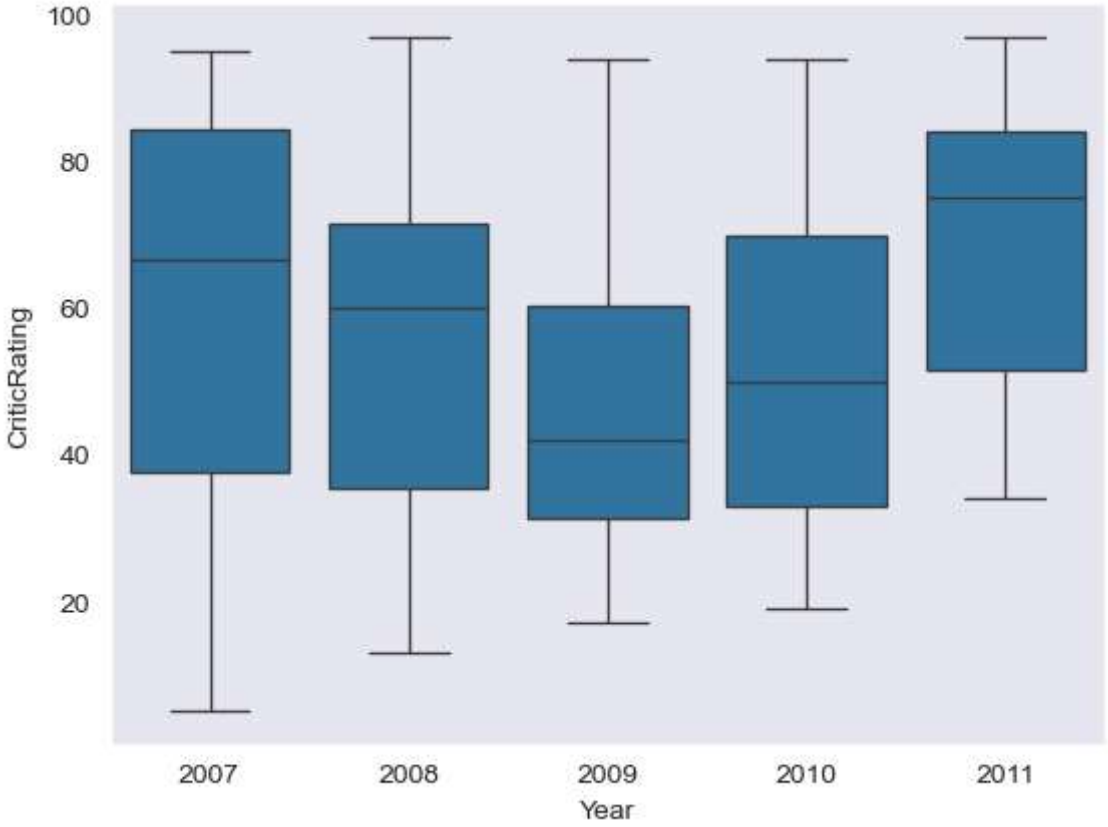
In [77]:
```
w = sns.boxplot(data=movies,x='Genre', y='CriticRating')
plt.show()
```



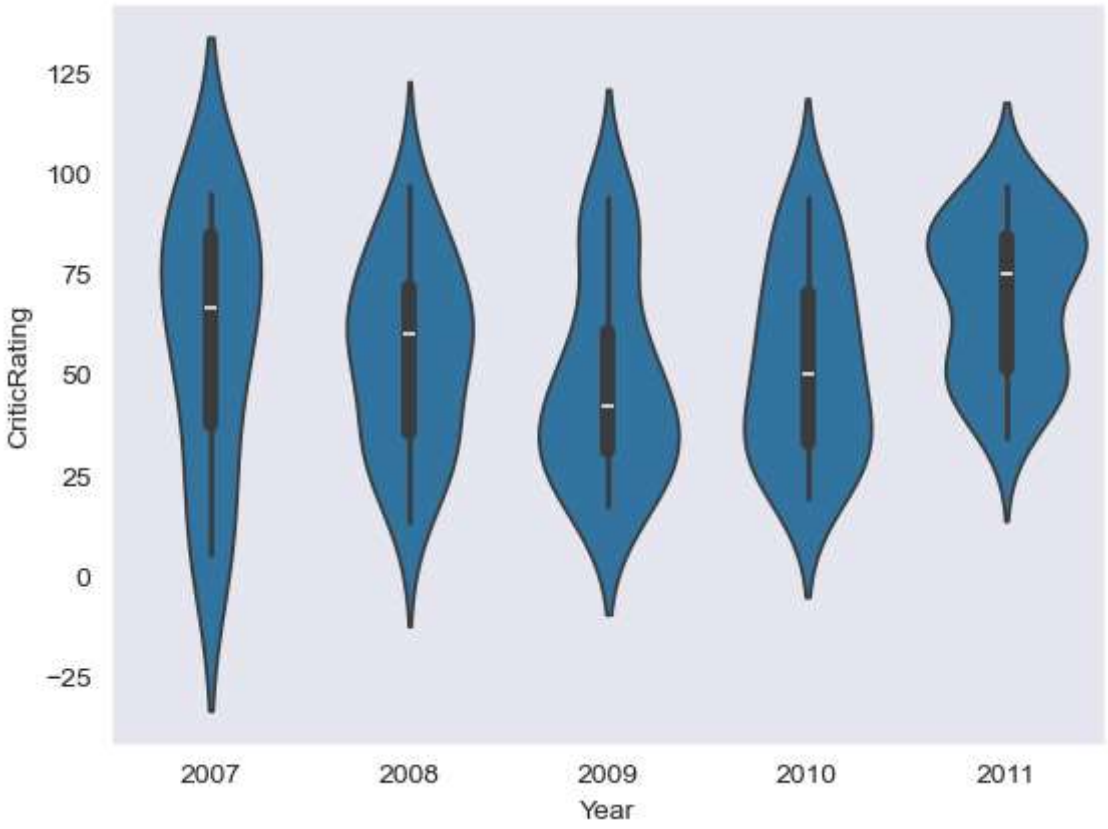
In [78]:
```
z = sns.violinplot(data=movies,x='Genre', y='CriticRating')
plt.show()
```



In [79]:
```
w1 = sns.boxplot(data=movies[movies.Genre=='Drama'],x='Year', y='CriticRating')
plt.show()
```

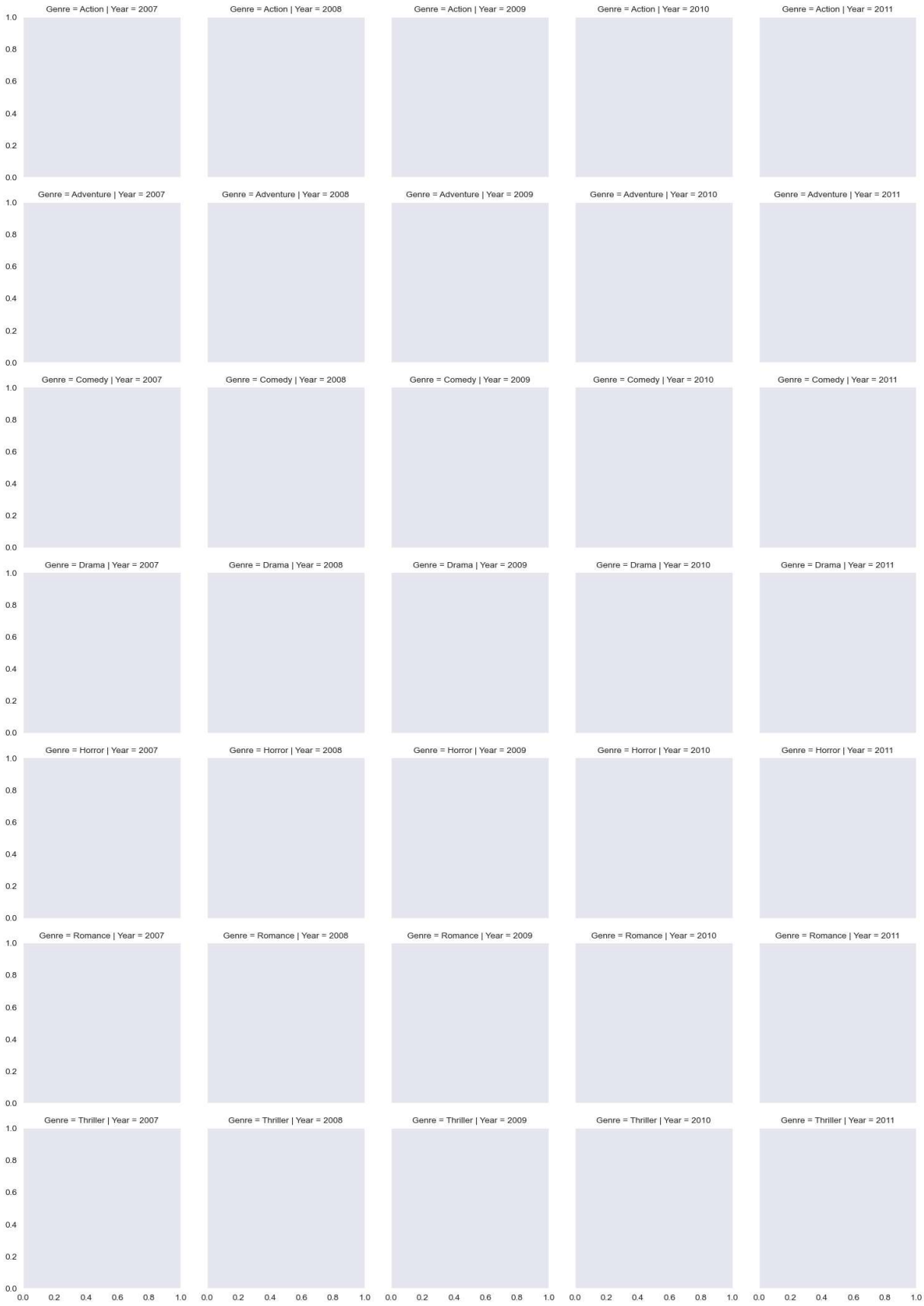


In [80]:
```
z1 = sns.violinplot(data=movies[movies.Genre=='Drama'],x='Year', y='CriticRating')
plt.show()
```

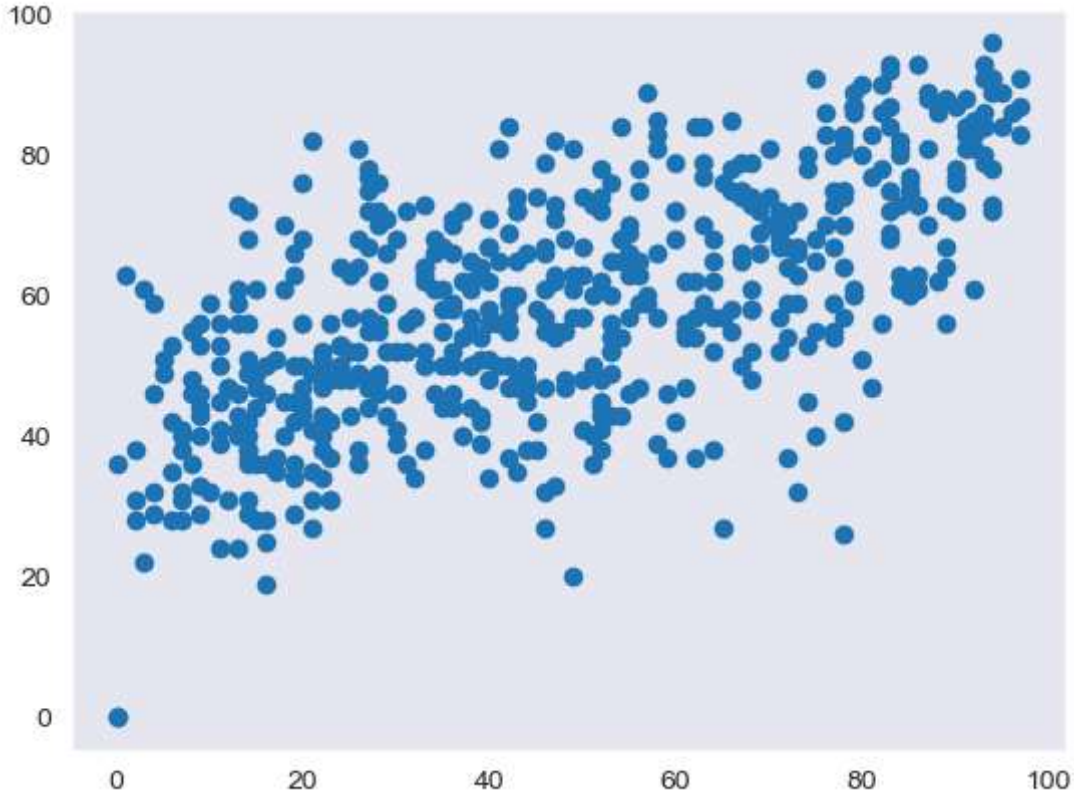


In [81]:
```
# Creating Facetgrid

# univariate(only one parameter)Analysis
# bivariate(two parameter)Analysis
# multivariate( multiple parameter)Analysis: use facetgrid
```

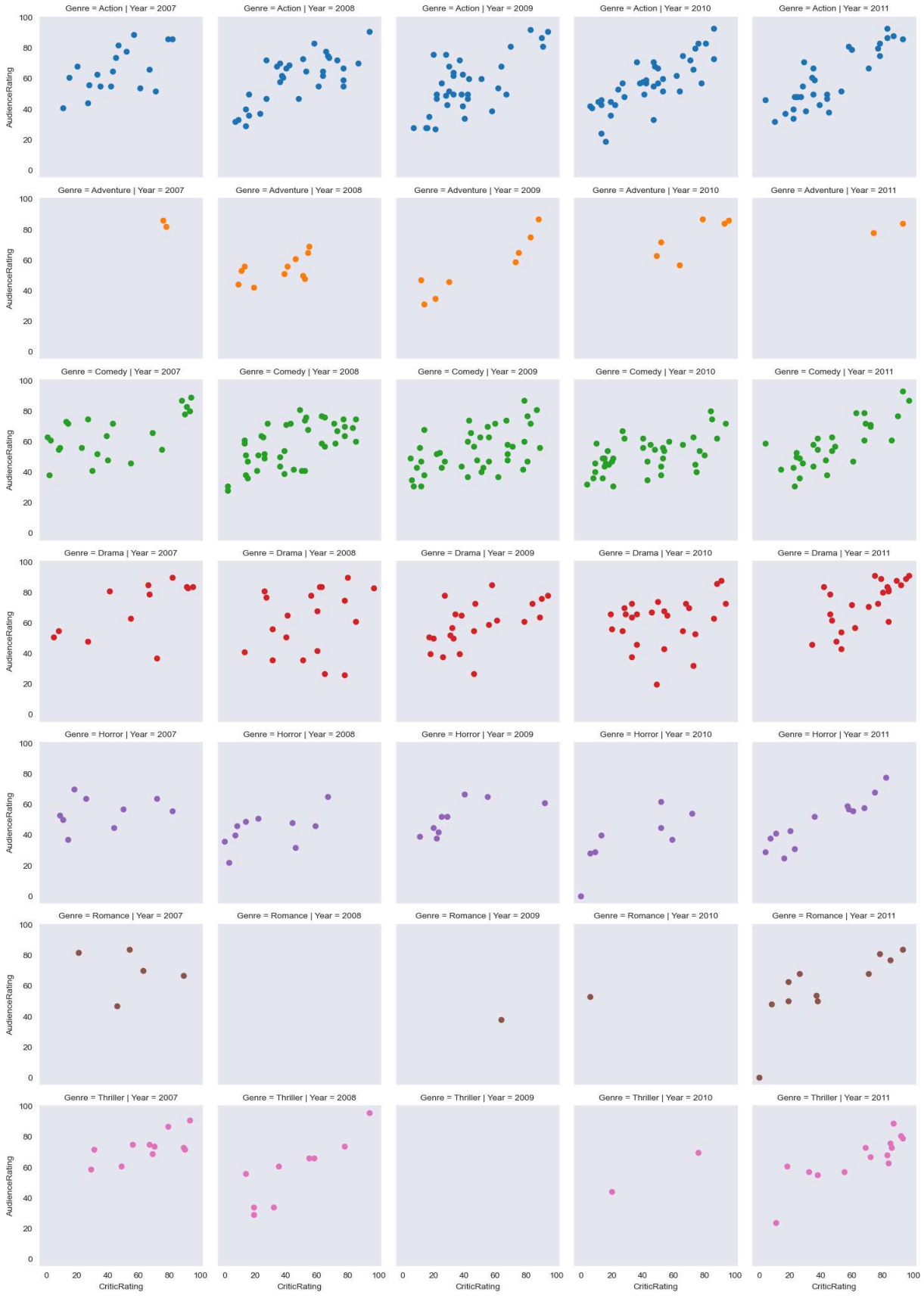
In [82]:
```
# creating a facetgrid
g= sns.FacetGrid(movies, row ='Genre', col ='Year', hue ='Genre')
plt.show()
```



In [83]:
```
plt.scatter(x=movies.CriticRating, y=movies.AudienceRating)
plt.show()
```



In [84]:
```
g = sns.FacetGrid(movies, row ='Genre', col ='Year', hue ='Genre')
g1 = g.map(plt.scatter, 'CriticRating', 'AudienceRating')
plt.show()
# scatterplot are mapped in facetgrid
```

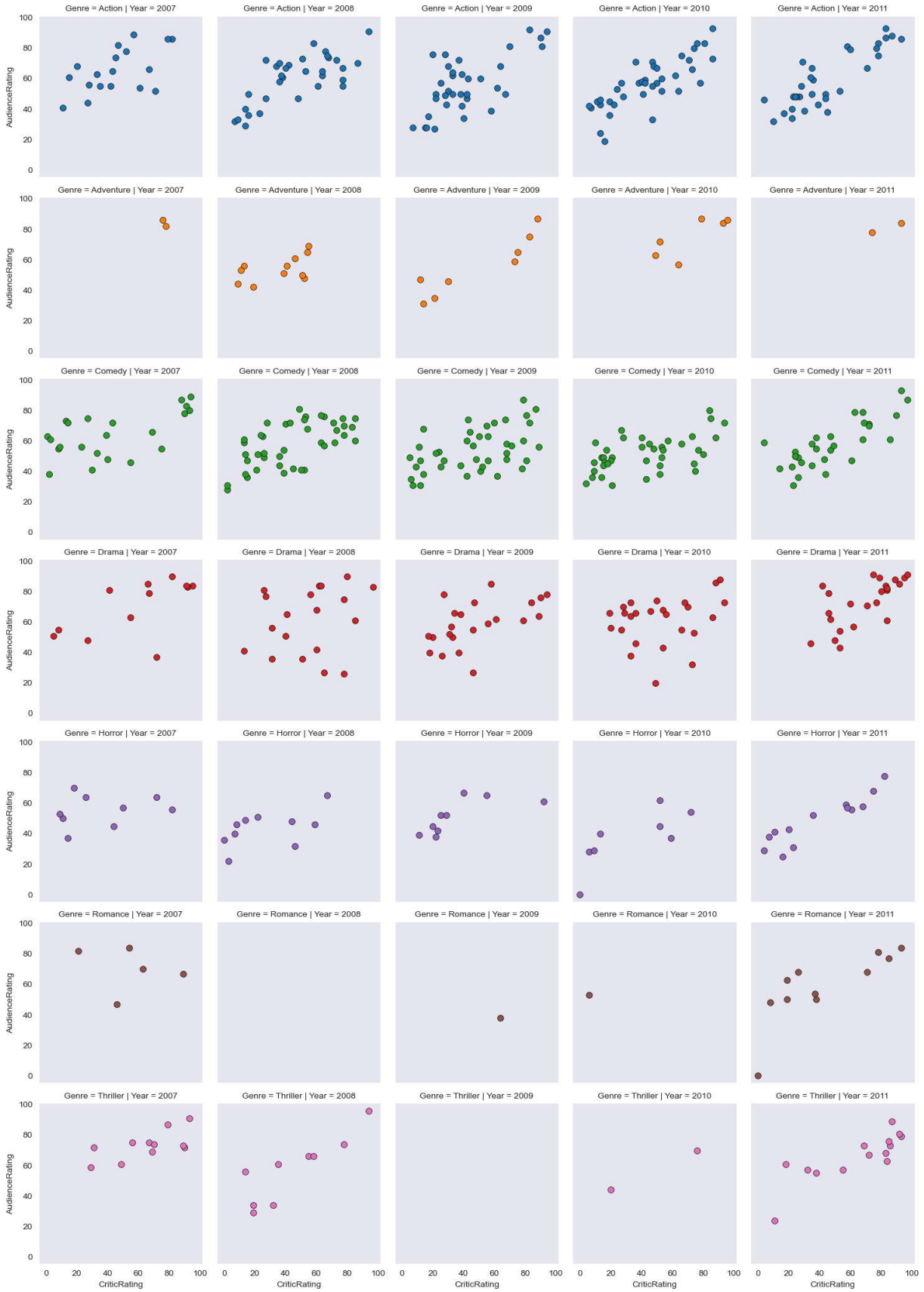


In [85]:
```
# you can populate any type of chart

g = sns.FacetGrid(movies, row ='Genre', col ='Year', hue ='Genre')
```

```
g2 = g.map(plt.hist, 'BudgetMillions')
plt.show()
```

In [86]:
```
g = sns.FacetGrid(movies, row ='Genre', col ='Year', hue ='Genre')
```

```
kws = dict(s=50, linewidth=0.5, edgecolor='black')
g3 = g.map(plt.scatter, 'CriticRating', 'AudienceRating', **kws)
plt.show()
```

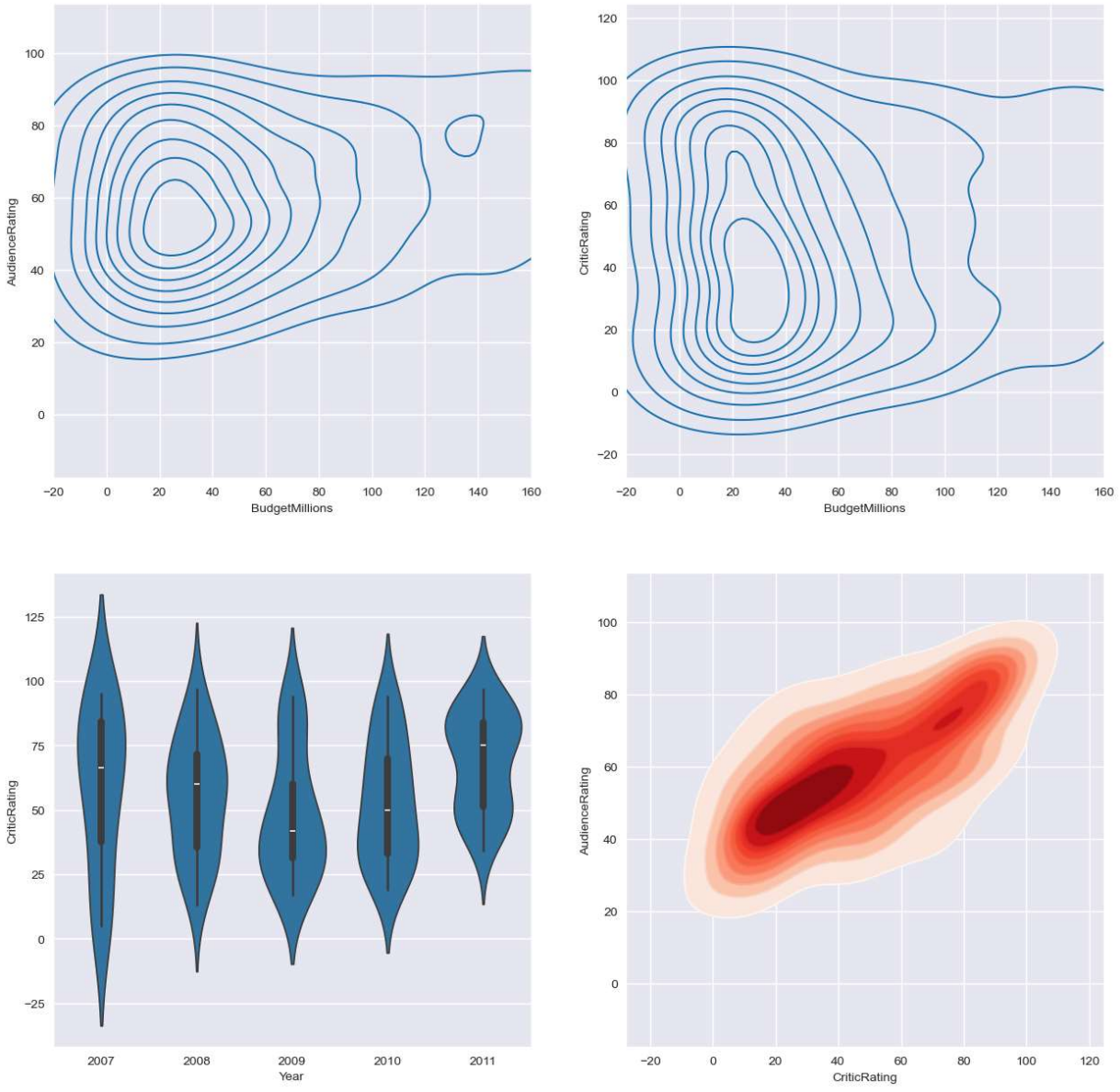


python is not vectorize programming language; but python supports vectorized programming through powerful libraries like NumPy,pandas # ex of vectorized programming: a=[1,2,3]; b=[4,5,6];then we can write a loop to perform addition of a+b # but by

vectorization can do it as result=(a+b); result == [5,7,9] ;instead of loops it perform operation on whole arrays

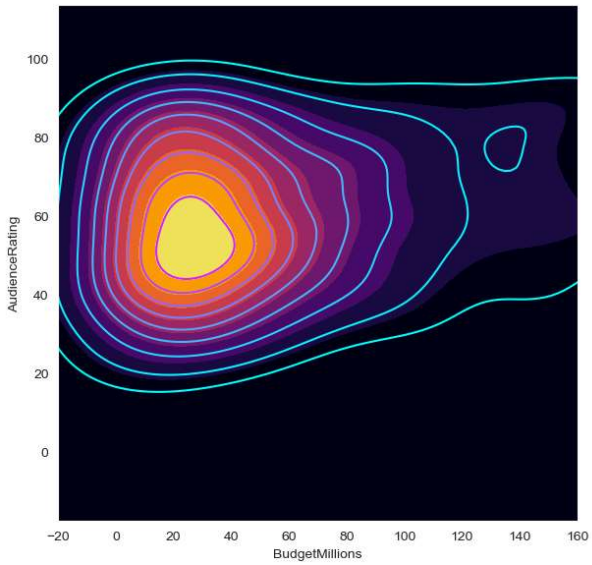
In [119…

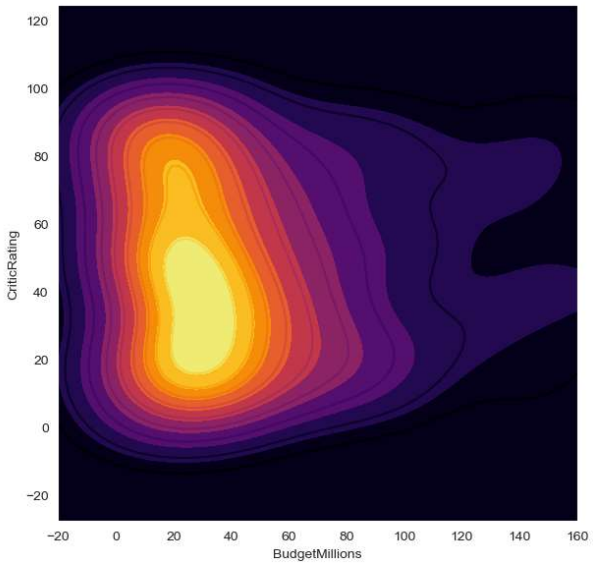
```
# Creating Dashboard(Dashboard- Combination of charts)

sns.set_style('darkgrid')
f,axes = plt.subplots(2,2,figsize = (15,15))

k1 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating, ax=axes[0,0])
k2 = sns.kdeplot(x=movies.BudgetMillions, y=movies.CriticRating, ax=axes[0,1])

k1.set(xlim=(-20,160))
k2.set(xlim=(-20,160))

z = sns.violinplot(data=movies[movies.Genre=='Drama'],x='Year',y='CriticRating', ax

k4 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, shade=True, shade_

k4b = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, cmap='Reds',ax=ax

plt.show()
```

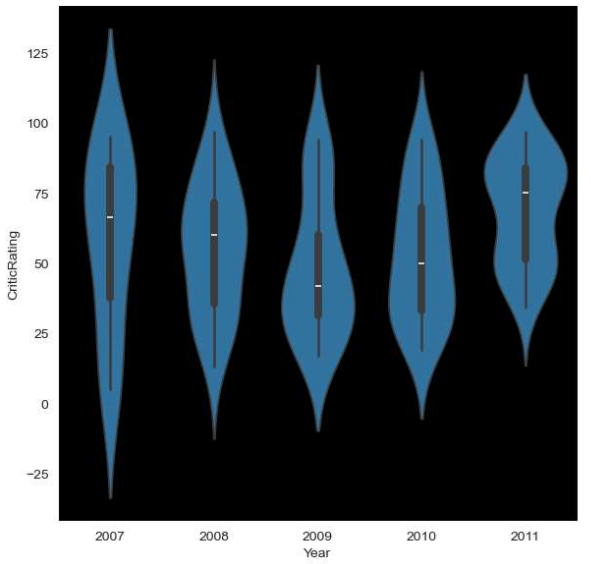


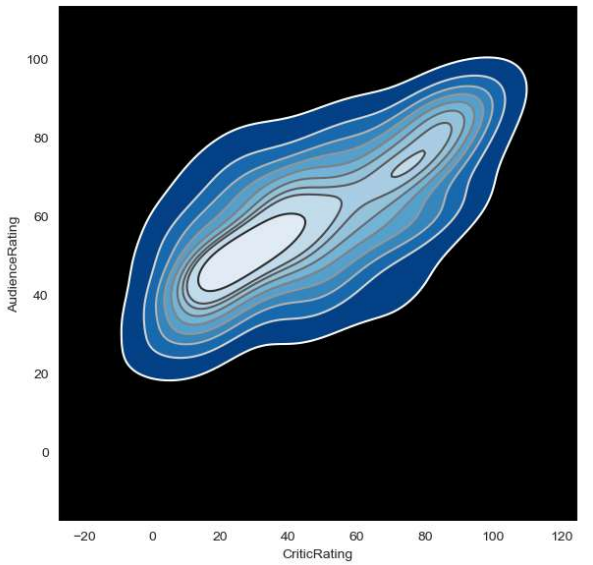
In [143…

```
# Building Dashboard
# How to style dashboard using different color
# here plots are overlayed (like k1 and k1b are two density plot with different col
# it's common in dashboard to layer plot for richer visuals.

sns.set_style('dark',{'axes.facecolor':'black'})
f,axes = plt.subplots (2,2,figsize = (15,15))

#plot[0,0]
k1 = sns.kdeplot(x=movies.BudgetMillions, y= movies.AudienceRating, \
                 shade =True,shade_lowest=True,cmap='inferno', \
                ax = axes[0,0])
k1b = sns.kdeplot(x=movies.BudgetMillions, y= movies.AudienceRating, \
    cmap='cool',ax=axes[0,0])

# plot[0,1]
k2 = sns.kdeplot(x=movies.BudgetMillions,y=movies.CriticRating, \
                 shade=True,shade_lowest=True, cmap='inferno',\
                 ax=axes[0,1])

k2 = sns.kdeplot(x=movies.BudgetMillions,y=movies.CriticRating, \
                 cmap='inferno', ax=axes[0,1])

#plot[1,0]
z = sns.violinplot(data=movies[movies.Genre=='Drama'], \
    x='Year', y='CriticRating', ax=axes[1,0])

#plot[1,1]
k4 = sns.kdeplot(x=movies.CriticRating, y= movies.AudienceRating, \
    shade=True, shade_lowest=False, cmap='Blues_r', \
    ax=axes[1,1])

k4 = sns.kdeplot(x=movies.CriticRating, y= movies.AudienceRating, \
    cmap='gist_gray_r', ax=axes[1,1])

k1.set(xlim=(-20,160))
k2.set(xlim=(-20,160))
plt.show()


# k1.set(xlim=(-20,160)); k2.set(xlim=(-20,160)) --->show the x-axis range from -20
# when multiple subplot are used ,aligning axes gives a better comparison between p
```









In [147…
```
movies.columns
```

Out[147…
```
Index(['Film', 'Genre', 'CriticRating', 'AudienceRating', 'BudgetMillions',
       'Year'],
      dtype='object')
```

In [199…
```
# practicing dashboard
# Your client is a film producer and want you to analyse a data and suggest him whi
sns.set_style('darkgrid')
f,axis= plt.subplots(2,2,figsize=(15,15))

p1 = sns.barplot(data=movies,x='Year',y='AudienceRating',hue='Genre',ax=axis[0,0])
plt.title('Yearly AudienceRating by Genre')

p2 = sns.kdeplot(x= movies.CriticRating, y=movies.AudienceRating,cmap='Greens', \
                 fill=True,ax=axes[0,1])
plt.title('plot')

p3 = sns.violinplot(y=movies.CriticRating,x=movies.Genre, ax=axis[1,0])
plt.legend()
```

```
plt.title('CriticeRating')

p4 = sns.boxplot(x=movies.Genre , y= movies.AudienceRating ,ax=axis[1,1])
plt.title('AudienceRating for different Genre Movies')

plt.suptitle('Movies Dataset Analysis')
plt.tight_layout()      # avoid overlappping of plots
plt.show()
# need to write= after figsize
# barplot do not support stacked argument
# sns do not have hist; it is present in plt
# kde plot use numerical value not categorical
# histplot is best for numeric data; for categorical data avoid using it
# in hist x and y both axis value can't be given at same time; instaed it have weig
# if use sns.histplot() argument multiple='stack' ;if using plt.hist() argument sta
# piechart do not have argument hue; pie chart used to give data over single variab
```